

```
In [27]: import sys
import os
import torch
import pandas as pd
from sklearn.model_selection import StratifiedShuffleSplit

print("Python usato dal notebook:", sys.executable)
print("Cartella corrente:", os.getcwd())
print("Pandas version:", pd.__version__)
print("CUDA disponibile:", torch.cuda.is_available())

if torch.cuda.is_available():
    print("GPU:", torch.cuda.get_device_name(0))
```

Python usato dal notebook: /usr/bin/python3
 Cartella corrente: /mnt/sdc1/2026_mlinf_gr03
 Pandas version: 2.3.3
 CUDA disponibile: True
 GPU: NVIDIA A100-SXM4-40GB MIG 1g.5gb

```
In [28]: import os

# Ora che sei sul server, il dataset è semplicemente la cartella locale
DATASET_PATH = "/home/2026_mlinf_gr03/waste_type_identification"

print(f"📁 Percorso attuale di lavoro: {os.getcwd()}")
print(f"📁 Il dataset verrà letto da: {DATASET_PATH}")

if os.path.exists(DATASET_PATH):
    print("✅ Cartella dataset agganciata sul server locale!")
    print("Sotto-cartelle:", os.listdir(DATASET_PATH))
```

📁 Percorso attuale di lavoro: /mnt/sdc1/2026_mlinf_gr03
 📁 Il dataset verrà letto da: /home/2026_mlinf_gr03/waste_type_identification
 ✅ Cartella dataset agganciata sul server locale!
 Sotto-cartelle: ['clothing', 'metal', 'battery', 'plastic', 'papery', 'undifferentiated', 'glass', 'organic']

1. Setup Iniziale, Seed e Mappatura Esegui questa cella per impostare la riproducibilità (Seed), definire i parametri modificabili e mappare la gerarchia delle cartelle. I parametri sono: dimensione dei batch, dimensione delle immagini, learning rate, numero epoche, quanto splittare training e val, il modello scelto, il metodo utilizzato (facciamo padding? randomsize?)

```
In [29]: import os
import random
import numpy as np
import torch

# =====
# 1. IMPOSTAZIONE SEED (Riproducibilità -
# =====

def set_seed(seed=42):
    random.seed(seed)
    os.environ['PYTHONHASHSEED'] = str(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
```

```

torch.cuda.manual_seed(seed)
torch.cuda.manual_seed_all(seed)
# Forza PyTorch ad usare algoritmi deterministici (rallenta leggermente ma g
torch.backends.cudnn.deterministic = True
torch.backends.cudnn.benchmark = False

# Cambia pure il seed se vuoi testare la robustezza in seguito
SEED = 42
set_seed(SEED)
print(f"--> Seed {SEED} impostato con successo.")

# =====
# 2. IPERPARAMETRI CONFIGURABILI
# =====
CONFIG = {
    # Risorse e hardware
    "device": torch.device("cuda" if torch.cuda.is_available() else "cpu"),

    # Parametri di addestramento
    "batch_size": 16,          # Modificabile (es. 16, 32, 64)
    "img_size": (224, 224),    # Dimensione standard per iniziare
    "lr": 1e-4,                # Learning rate iniziale per la baseline
    "epochs": 10,              # Poche epoche per il primo test di "overfitting"
    "val_split": 0.2,          # Sotto-split interno (es. 80% train, 20% val)

    # Scelte di Architettura e Preprocessing
    "model_name": "efficientnet_b0", # Per ora partiamo da una CNN semplice (
    "preprocessing_mode": "padding", # Opzioni: "padding", "resize_only", "ran
}

# =====
# 3. MAPPATURA ADDESTRAMENTO (12 Classi distinte)
# =====
# Assegniamo un ID univoco da 0 a 11 a ogni singola cartella foglia del dataset
TRAIN_LABEL_MAP = {
    "battery": 0,
    "clothes": 1,
    "shoes": 2,
    "brown": 3,
    "green": 4,
    "transparent": 5,
    "metal": 6,
    "organic": 7,
    "cardboard": 8,
    "paper": 9,
    "plastic": 10,
    "undifferentiated": 11
}

# Inversione utile per fare i plot di controllo (es. mostrare "shoes" invece di
TRAIN_ID_TO_NAME = {v: k for k, v in TRAIN_LABEL_MAP.items()}

# =====
# 4. MAPPATURA UFFICIALE DI PROGETTO (8 Macro-classi)
# =====
TARGET_LABEL_MAP = {
    "Battery": 0,
    "Clothing": 1,
    "Glass": 2,
    "Metal": 3,

```

```

    "Organic": 4,
    "Papery": 5,
    "Plastic": 6,
    "Undifferentiated": 7
}

# =====
# 5. IL PONTE: Convertitore da 12 a 8 classi
# =====
# Questo dizionario prende l'ID a 12 classi e restituisce l'ID ufficiale a 8 cla
# Sarà fondamentale nella funzione predict() e nel calcolo della Balanced Accura
TRAIN_TO_TARGET_MAPPING = {
    TRAIN_LABEL_MAP["battery"]: TARGET_LABEL_MAP["Battery"],           # 0
    TRAIN_LABEL_MAP["clothes"]: TARGET_LABEL_MAP["Clothing"],           # 1
    TRAIN_LABEL_MAP["shoes"]: TARGET_LABEL_MAP["Clothing"],             # 2
    TRAIN_LABEL_MAP["brown"]: TARGET_LABEL_MAP["Glass"],                # 3
    TRAIN_LABEL_MAP["green"]: TARGET_LABEL_MAP["Glass"],                # 4
    TRAIN_LABEL_MAP["transparent"]: TARGET_LABEL_MAP["Glass"],          # 5
    TRAIN_LABEL_MAP["metal"]: TARGET_LABEL_MAP["Metal"],                 # 6
    TRAIN_LABEL_MAP["organic"]: TARGET_LABEL_MAP["Organic"],             # 7
    TRAIN_LABEL_MAP["cardboard"]: TARGET_LABEL_MAP["Papery"],            # 8
    TRAIN_LABEL_MAP["paper"]: TARGET_LABEL_MAP["Papery"],                # 9
    TRAIN_LABEL_MAP["plastic"]: TARGET_LABEL_MAP["Plastic"],             # 10
    TRAIN_LABEL_MAP["undifferentiated"]: TARGET_LABEL_MAP["Undifferentiated"] # 11
}

# Aggiorniamo la configurazione iniziale per la rete neurale
CONFIG["num_classes_train"] = 12
CONFIG["num_classes_target"] = 8

print(f"--> Setup completato: la rete uscirà con {CONFIG['num_classes_train']} c
print(f"--> Pronta la conversione automatica verso i {CONFIG['num_classes_target']}

--> Seed 42 impostato con successo.
--> Setup completato: la rete uscirà con 12 canali per l'addestramento.
--> Pronta la conversione automatica verso i 8 target di valutazione.

```

2. Pipeline di Preprocessing (Padding vs RandomResize) Ecco la funzione di trasformazione personalizzata per applicare il Padding quadrato (o il random resize?) mantenendo intatto l'aspect ratio originale per non tagliare o deformare i soggetti lunghi/stretti o posizionati sui bordi.

```

In [30]: import glob
from PIL import Image
from torchvision import transforms
from sklearn.model_selection import StratifiedShuffleSplit

# Definizione del percorso principale del dataset su Google Drive
#DATASET_PATH = "/content/dataset_locale"

# =====
# CONFIGURAZIONE AGGIUNTIVA PER PREPROCESSING & DATA AUGMENTATION
# =====
# Aggiungiamo un interruttore booleano per l'augmentation alla CONFIG esistente
# Cambialo in True più avanti quando vorrai attivare le modifiche casuali sul Tr
CONFIG["use_augmentation"] = False

```

```

print(f"--> Percorso Dataset impostato: {DATASET_PATH}")
print(f"--> Stato iniziale Augmentation: {CONFIG['use_augmentation']}")
print(f"--> Metodo Preprocessing scelto: {CONFIG['preprocessing_mode']}")

# =====
# FUNZIONE DI UTILITÀ PER IL PADDING (Mantiene le proporzioni originali)
# =====
def pad_to_square(image):
    """
    Prende un'immagine PIL e aggiunge del padding nero sui bordi
    per farla diventare quadrata, senza distorcere l'oggetto.
    """
    w, h = image.size
    max_wh = max(w, h)
    hp = (max_wh - w) // 2
    vp = (max_wh - h) // 2
    # Calcola il padding per i 4 lati (sinistra, sopra, destra, sotto)
    padding = (hp, vp, max_wh - w - hp, max_wh - h - vp)
    return transforms.functional.pad(image, padding, fill=0, padding_mode='const')

# =====
# BLOCCO SEPARATO: GENERATORE DINAMICO DI TRASFORMAZIONI (Flessibile)
# =====
def get_transforms_pipeline(config, is_training_set=True):
    """
    Costruisce la pipeline di trasformazioni in base alle scelte nella CONFIG.
    Permette di attivare/disattivare o cambiare modalità al volo.
    """
    pipeline_steps = []

    # 1. Gestione del Preprocessing (Padding vs Solo Resize vs Random Crop vs Or
    if config["preprocessing_mode"] == "padding":
        # Prima rende l'immagine quadrata con il padding, poi ridimensiona
        pipeline_steps.append(transforms.Lambda(lambda img: pad_to_square(img)))
        pipeline_steps.append(transforms.Resize(config["img_size"]))

    elif config["preprocessing_mode"] == "resize_only":
        # Forza il ridimensionamento schiacciando l'immagine se non è quadrata
        pipeline_steps.append(transforms.Resize(config["img_size"]))

    elif config["preprocessing_mode"] == "random_crop":
        # SE È IL TRAIN SET: ritaglia una porzione casuale e la ridimensiona a (
        if is_training_set:
            pipeline_steps.append(transforms.RandomResizedCrop(config["img_size"]
        # SE È IL VAL SET: NON possiamo usare il ritaglio casuale (cambiarebbe a
        # Usiamo CenterCrop (ritaglio centrale) o Resize standard per mantenere
        else:
            pipeline_steps.append(transforms.Resize(config["img_size"]))
            pipeline_steps.append(transforms.CenterCrop(config["img_size"]))

    elif config["preprocessing_mode"] == "original":
        # Non aggiunge modifiche dimensionali
        pass

    # 2. Gestione Data Augmentation (Attiva solo se richiesto E solo sul Trainin
    if config["use_augmentation"] and is_training_set:
        pipeline_steps.append(transforms.RandomHorizontalFlip(p=0.5))
        pipeline_steps.append(transforms.RandomRotation(degrees=15))

    # 3. Trasformazioni strutturali obbligatorie per dare i dati in pasto alla r

```

```

pipeline_steps.append(transforms.ToTensor())
# Normalizzazione standard (ImageNet weights) per aiutare la convergenza
pipeline_steps.append(transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[

return transforms.Compose(pipeline_steps)

```

```

--> Percorso Dataset impostato: /home/2026_mlinf_gr03/waste_type_identification
--> Stato iniziale Augmentation: False
--> Metodo Preprocessing scelto: padding

```

3. Split del Dataset Stratificato e Salvataggio CSV Questo codice scansiona le cartelle, effettua uno split stratificato sulle sotto-classi (garantendo un bilanciamento perfetto tra train e validation) e genera il file .csv richiesto per la consegna. Il salvataggio csv la aggiungiamo alla consegna, sennò ci riempiamo di file csv che non ci servono!!!! Visto che split è un iper-parametro, lo possiamo cambiare per trovare quello migliore per il dataset

```

In [31]: from collections import Counter
import pandas as pd
import os
import glob
import torch
import random
from PIL import Image

# =====
# CSV DELLO SPLIT: NON VIENE PIÙ CANCELLATO AUTOMATICAMENTE
# =====

SPLIT_CSV_PATH = f"split_protocol_seed{SEED}_val{int(CONFIG['val_split'] * 100)}"

print(f"--> File CSV dello split impostato su: {SPLIT_CSV_PATH}")

TARGET_ID_TO_NAME = {v: k for k, v in TARGET_LABEL_MAP.items()}
# =====
# CLASSE DATASET CUSTOM
# =====

class WasteDataset(torch.utils.data.Dataset):
    def __init__(self, root_dir, image_paths=None, labels_12=None, transform=None):
        self.root_dir = root_dir
        self.transform = transform

        if image_paths is not None and labels_12 is not None:
            self.image_paths = image_paths
            self.labels_12 = labels_12
        else:
            self.image_paths = []
            self.labels_12 = []
            self._load_dataset()

    def _load_dataset(self):
        extensions = ('*.jpg', '*.jpeg', '*.png', '*.JPG', '*.JPEG', '*.PNG')

        for ext in extensions:
            search_pattern = os.path.join(self.root_dir, "**", ext)

            for filepath in glob.glob(search_pattern, recursive=True):
                folder_name = os.path.basename(os.path.dirname(filepath))

```

```

        if folder_name in TRAIN_LABEL_MAP:
            self.image_paths.append(filepath)
            self.labels_12.append(TRAIN_LABEL_MAP[folder_name])

# Ordina i file per rendere lo split più stabile insieme al seed
combined = sorted(zip(self.image_paths, self.labels_12), key=lambda x: x

if len(combined) == 0:
    raise ValueError("Nessuna immagine trovata. Controlla DATASET_PATH e

self.image_paths, self.labels_12 = zip(*combined)
self.image_paths = list(self.image_paths)
self.labels_12 = list(self.labels_12)

def __len__(self):
    return len(self.image_paths)

def __getitem__(self, idx):
    tentativi = 0
    current_idx = idx

# Tentativi massimi per evitare loop infiniti se l'intero dataset fosse
    while tentativi < 15:
        try:
            img_path = self.image_paths[current_idx]

            # Caricamento lazy dell'immagine + conversione RGB forzata (riso
            image = Image.open(img_path).convert("RGB")

            # Label originale a 12 sotto-classi
            label_12 = self.labels_12[current_idx]

            # Label ufficiale a 8 macro-classi
            label_8 = TRAIN_TO_TARGET_MAPPING[label_12]

            if self.transform:
                image = self.transform(image)

            # Se tutto ha successo, restituisce i dati ed esce dal __getitem
            return image, label_12, label_8

        except Exception as e:
            # STAMPA DI ERRORE: Ti avvisa immediatamente a schermo quando sa
            print(f"\n⚠ [TRY-CATCH ATTIVATO] File corrotto o illeggibile a
            print(f"    └─ Percorso file: {self.image_paths[current_idx]}")
            print(f"    └─ Errore catturato: {e}")
            print(f"    └─ Azione: Scarto il file e peso un indice casuale

            # Cambiamo l'indice prendendone uno completamente a caso dal dat
            current_idx = random.randint(0, len(self.image_paths) - 1)
            tentativi += 1

# Fallback nel caso estremo in cui fallisca per 15 immagini di fila
    raise RuntimeError(f"Impossibile caricare i file. Rilevate troppe immagini

# =====
# FUNZIONE DI STAMPA DISTRIBUZIONE
# =====

```

```

def print_distribution(title, train_labels, val_labels, id_to_name):
    train_counts = Counter(train_labels)
    val_counts = Counter(val_labels)

    print(f"\n--> {title}")
    print(f"{'Classe':<20} | {'Train n':<8} | {'Train %':<10} | {'Val n':<8} | {'Val %':<10}")
    print("-" * 75)

    for cls_id in sorted(id_to_name.keys()):
        train_n = train_counts[cls_id]
        val_n = val_counts[cls_id]

        train_p = (train_n / len(train_labels)) * 100
        val_p = (val_n / len(val_labels)) * 100

        print(f"{id_to_name[cls_id]:<20} | {train_n:<8} | {train_p:>6.2f}% | {val_n:<8} | {val_p:>6.2f}%")

# =====
# STEP 1: SCANSIONE DEL DATASET
# =====
print("\n--> Step 1: Scansione del Drive per mappare i file...")
base_dataset = WasteDataset(root_dir=DATASET_PATH)

print(f"    Immagini totali trovate: {len(base_dataset)}")

# =====
# STEP 2: CREAZIONE O CARICAMENTO DELLO SPLIT CSV
# =====
if os.path.exists(SPLIT_CSV_PATH):
    print(f"\n--> Step 2: CSV dello split già esistente trovato.")
    print(f"    Caricamento da: {SPLIT_CSV_PATH}")

    split_df = pd.read_csv(SPLIT_CSV_PATH)

    train_df = split_df[split_df["split"] == "train"]
    val_df = split_df[split_df["split"] == "val"]

    train_paths = train_df["filepath"].tolist()
    trainlbls = train_df["label_12_id"].astype(int).tolist()

    val_paths = val_df["filepath"].tolist()
    vallbls = val_df["label_12_id"].astype(int).tolist()
else:
    print("\n--> Step 2: Creazione nuovo split stratificato sulle 12 sotto-classi")

    labels = base_dataset.labels_12

    sss = StratifiedShuffleSplit(
        n_splits=1,
        test_size=CONFIG["val_split"],
        random_state=SEED
    )

    train_idx, val_idx = next(sss.split(X=range(len(labels)), y=labels))

    train_paths = [base_dataset.image_paths[i] for i in train_idx]
    trainlbls = [base_dataset.labels_12[i] for i in train_idx]

```

```

val_paths = [base_dataset.image_paths[i] for i in val_idx]
val_lbls = [base_dataset.labels_12[i] for i in val_idx]

# Creazione righe CSV
split_rows = []

for path, label_12 in zip(train_paths, train_lbls):
    label_8 = TRAIN_TO_TARGET_MAPPING[label_12]

    split_rows.append({
        "filepath": path,
        "split": "train",
        "label_12_id": label_12,
        "label_12_name": TRAIN_ID_TO_NAME[label_12],
        "label_8_id": label_8,
        "label_8_name": TARGET_ID_TO_NAME[label_8]
    })

for path, label_12 in zip(val_paths, val_lbls):
    label_8 = TRAIN_TO_TARGET_MAPPING[label_12]

    split_rows.append({
        "filepath": path,
        "split": "val",
        "label_12_id": label_12,
        "label_12_name": TRAIN_ID_TO_NAME[label_12],
        "label_8_id": label_8,
        "label_8_name": TARGET_ID_TO_NAME[label_8]
    })

split_df = pd.DataFrame(split_rows)
split_df.to_csv(SPLIT_CSV_PATH, index=False)

print(f"    Split creato e salvato in: {SPLIT_CSV_PATH}")

# =====
# STEP 3: RIEPILOGO NUMERICO DELLO SPLIT
# =====
print("\n[RISULTATO DELLO SPLIT]")
print(f"    Immagini totali trovate: {len(base_dataset)}")
print(f"    Immagini nel Train Set: {len(train_paths)}")
print(f"    Immagini nel Val Set: {len(val_paths)}")

# =====
# STEP 4: VERIFICA DISTRIBUZIONE SULLE 12 SOTTO-CLASSI
# =====
print_distribution(
    title="A. Distribuzione delle 12 SOTTO-CLASSI",
    train_labels=train_lbls,
    val_labels=val_lbls,
    id_to_name=TRAIN_ID_TO_NAME
)

# =====
# STEP 5: VERIFICA DISTRIBUZIONE SULLE 8 MACRO-CLASSI UFFICIALI
# =====

```



```

trainlbls_8 = [TRAIN_TO_TARGET_MAPPING[lbl] for lbl in trainlbls]
vallbls_8 = [TRAIN_TO_TARGET_MAPPING[lbl] for lbl in vallbls]

print_distribution(
    title="B. Distribuzione delle 8 MACRO-CLASSI UFFICIALI",
    train_labels=trainlbls_8,
    val_labels=vallbls_8,
    id_to_name=TARGET_ID_TO_NAME
)

# =====
# STEP 6: GENERAZIONE DELLE TRASFORMAZIONI
# =====
train_transforms = get_transforms_pipeline(CONFIG, is_training_set=True)
val_transforms = get_transforms_pipeline(CONFIG, is_training_set=False)

# =====
# STEP 7: CREAZIONE DEI DATASET FINALI
# =====
train_dataset = WasteDataset(
    root_dir=DATASET_PATH,
    image_paths=train_paths,
    labels_12=trainlbls,
    transform=train_transforms
)

val_dataset = WasteDataset(
    root_dir=DATASET_PATH,
    image_paths=val_paths,
    labels_12=vallbls,
    transform=val_transforms
)

print("\n--> Dataset finali creati correttamente con protezione antiblocco attiv
print(f"    Train dataset: {len(train_dataset)} immagini")
print(f"    Val dataset:    {len(val_dataset)} immagini")

```

--> File CSV dello split impostato su: split_protocol_seed42_val20.csv

--> Step 1: Scansione del Drive per mappare i file...
Immagini totali trovate: 15515

--> Step 2: CSV dello split già esistente trovato.
Caricamento da: split_protocol_seed42_val20.csv

[RISULTATO DELLO SPLIT]

Immagini totali trovate: 15515
Immagini nel Train Set: 12412
Immagini nel Val Set: 3103

--> A. Distribuzione delle 12 SOTTO-CLASSI

Classe	Train n	Train %	Val n	Val %
battery	756	6.09%	189	6.09%
clothes	4260	34.32%	1065	34.32%
shoes	1582	12.75%	395	12.73%
brown	485	3.91%	122	3.93%
green	503	4.05%	126	4.06%
transparent	620	5.00%	155	5.00%
metal	615	4.95%	154	4.96%
organic	788	6.35%	197	6.35%
cardboard	713	5.74%	178	5.74%
paper	840	6.77%	210	6.77%
plastic	692	5.58%	173	5.58%
undifferentiated	558	4.50%	139	4.48%

--> B. Distribuzione delle 8 MACRO-CLASSI UFFICIALI

Classe	Train n	Train %	Val n	Val %
Battery	756	6.09%	189	6.09%
Clothing	5842	47.07%	1460	47.05%
Glass	1608	12.96%	403	12.99%
Metal	615	4.95%	154	4.96%
Organic	788	6.35%	197	6.35%
Papery	1553	12.51%	388	12.50%
Plastic	692	5.58%	173	5.58%
Undifferentiated	558	4.50%	139	4.48%

--> Dataset finali creati correttamente con protezione antiblocco attiva.

Train dataset: 12412 immagini
Val dataset: 3103 immagini

4. Dataset e Batching (DataLoaders) I Batch vengono generati in questa fase. Il DataLoader prende le immagini singole dal modulo WasteDataset, le raggruppa in blocchi di dimensione pari a BATCH_SIZE e si occupa di inviarle alla GPU in modo efficiente.

```
In [32]: import matplotlib.pyplot as plt
import torchvision
import numpy as np

# =====
# 1. CREAZIONE DEI DATALOADERS (Train e Validation)
# =====
NUM_WORKERS = 4
```

```

PIN_MEMORY = True if torch.cuda.is_available() else False

train_loader = torch.utils.data.DataLoader(
    train_dataset,
    batch_size=CONFIG["batch_size"],
    shuffle=True,
    num_workers=NUM_WORKERS,
    pin_memory=PIN_MEMORY
)

val_loader = torch.utils.data.DataLoader(
    val_dataset,
    batch_size=CONFIG["batch_size"],
    shuffle=False,
    num_workers=NUM_WORKERS,
    pin_memory=PIN_MEMORY
)

print("--> DataLoader creati con successo!")
print(f"    Batch Size configurato: {CONFIG['batch_size']}")
print(f"    Numero di batch nel Train Loader: {len(train_loader)} (Totale immagini
print(f"    Numero di batch nel Val Loader:    {len(val_loader)} (Totale immagini

# =====
# 2. ISPEZIONE DI SICUREZZA
# =====

images, labels_12, labels_8 = next(iter(train_loader))

print("\n[VERIFICA STRUTTURALE DEL BATCH]")
print(f"    Shape del batch di immagini: {images.shape}")
print(f"    Shape etichette 12 classi:    {labels_12.shape}")
print(f"    Shape etichette 8 classi:     {labels_8.shape}")

print("\n[CONTROLLO PRIME ETICHETTE]")
print(f"    Prime label 12 classi: {labels_12[:8].tolist()}")
print(f"    Prime label 8 classi:  {labels_8[:8].tolist()}")

# =====
# 3. FUNZIONE DI VISUALIZZAZIONE
# =====

def imshow_batch(img_tensor, labels_12_tensor, labels_8_tensor=None, max_images=
    """
    Mostra un batch di immagini e le relative etichette.
    Di default visualizza i nomi delle 12 sotto-classi.
    Se disponibili, mostra anche le macro-classi a 8 classi.
    """

    imgs_to_show = img_tensor[:max_images].cpu()
    labels_12_to_show = labels_12_tensor[:max_images].cpu()

    if labels_8_tensor is not None:
        labels_8_to_show = labels_8_tensor[:max_images].cpu()
    else:
        labels_8_to_show = None

    # Inversione normalizzazione ImageNet
    mean = torch.tensor([0.485, 0.456, 0.406]).view(3, 1, 1)
    std = torch.tensor([0.229, 0.224, 0.225]).view(3, 1, 1)

```

```

imgs_to_show = imgs_to_show * std + mean
imgs_to_show = torch.clamp(imgs_to_show, 0, 1)

grid = torchvision.utils.make_grid(imgs_to_show, nrow=4)
np_grid = grid.numpy()

plt.figure(figsize=(16, 10))
plt.imshow(np.transpose(np_grid, (1, 2, 0)))

class_names_12 = [
    TRAIN_ID_TO_NAME[label.item()]
    for label in labels_12_to_show
]

if labels_8_to_show is not None:
    class_names_8 = [
        TARGET_ID_TO_NAME[label.item()]
        for label in labels_8_to_show
    ]

    title = "Campioni estratti dal batch casuale\n"
    title += f"12 sotto-classi: {class_names_12}\n"
    title += f"8 macro-classi: {class_names_8}"
else:
    title = "Campioni estratti dal batch casuale\n"
    title += f"12 sotto-classi: {class_names_12}"

plt.title(title, fontsize=12, fontweight='bold')
plt.axis('off')
plt.show()

# =====
# 4. VISUALIZZAZIONE DI CONTROLLO
# =====
print("\n--> Generazione della griglia visiva di controllo...")
imshow_batch(images, labels_12, labels_8, max_images=8)

```

```

--> DataLoader creati con successo!
Batch Size configurato: 16
Numero di batch nel Train Loader: 776 (Totale immagini: 12412)
Numero di batch nel Val Loader: 194 (Totale immagini: 3103)

```

```

[VERIFICA STRUTTURALE DEL BATCH]
Shape del batch di immagini: torch.Size([16, 3, 224, 224])
Shape etichette 12 classi: torch.Size([16])
Shape etichette 8 classi: torch.Size([16])

```

```

[CONTROLLO PRIME ETICHETTE]
Prime label 12 classi: [6, 1, 1, 6, 3, 5, 2, 0]
Prime label 8 classi: [3, 1, 1, 3, 2, 2, 1, 0]

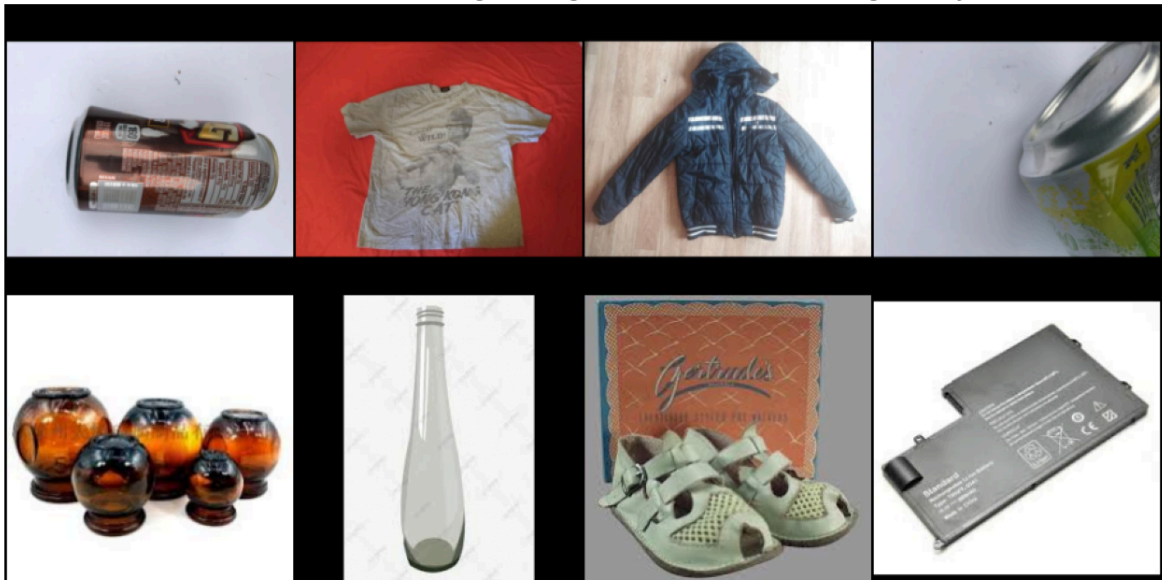
```

```

--> Generazione della griglia visiva di controllo...

```

Campioni estratti dal batch casuale
 12 sotto-classi: ['metal', 'clothes', 'clothes', 'metal', 'brown', 'transparent', 'shoes', 'battery']
 8 macro-classi: ['Metal', 'Clothing', 'Clothing', 'Metal', 'Glass', 'Glass', 'Clothing', 'Battery']



5. questo blocco definisce l'architettura della nostra rete SimpleCNN di baseline, strutturata su tre blocchi convolutivi con Batch Normalization, Max Pooling e Dropout per garantire stabilità e controllo del sovraddestramento. Il modello elabora le immagini RGB a tre canali per mappare le feature apprese su 12 classi di uscita distinte. Al termine della definizione, un test di sicurezza strutturale convalida la correttezza matematica delle dimensioni dei tensori simulando il passaggio di un batch direttamente sul dispositivo hardware selezionato.

```
In [33]: import torch
import torch.nn as nn
import torchvision.models as models

# =====
# DEFINIZIONE DELL'ARCHITETTURA REALE (Transfer Learning con EfficientNet)
# =====
class WasteNet(nn.Module):
    def __init__(self, num_classes=12, pretrained=True):
        super(WasteNet, self).__init__()

        # 1. Carichiamo EfficientNet-B0 con i pesi ufficiali pre-addestrati su I
        if pretrained:
            # Sintassi moderna richiesta dalle ultime versioni di torchvision
            weights = models.EfficientNet_B0_Weights.DEFAULT
            self.backbone = models.efficientnet_b0(weights=weights)
            print("--> EfficientNet-B0 caricata con pesi pre-addestrati (ImageNe
        else:
            self.backbone = models.efficientnet_b0(weights=None)
            print("--> EfficientNet-B0 caricata da zero (senza pesi).")

        # 2. Estraiamo il numero di feature in ingresso al classificatore origin
        # Per EfficientNet-B0, l'ultimo strato lineare (self.backbone.classifier
        num_features = self.backbone.classifier[1].in_features

        # 3. Sostituiamo la testa di classificazione (Classifier) adattandola al
        # Manteniamo un Dropout adeguato (p=0.4) per evitare l'overfitting dato
        self.backbone.classifier = nn.Sequential(
            nn.Dropout(p=0.4, inplace=True),
```

```

        nn.Linear(in_features=num_features, out_features=num_classes)
    )
    print(f"--> Testa di classificazione adattata: output impostato su {num_

def forward(self, x):
    # Il passaggio in avanti sfrutta l'estrazione di feature profonda di Eff
    return self.backbone(x)

# =====
# INIZIALIZZAZIONE E TEST STRUTTURALE DEL MODELLO VERO
# =====
# Creiamo l'istanza di WasteNet usando le 12 classi salvate nel CONFIG
model = WasteNet(num_classes=CONFIG["num_classes_train"], pretrained=True)

# Spostiamo il modello sulla risorsa hardware corretta (GPU Cuda)
model = model.to(CONFIG["device"])

print(f"--> Modello '{CONFIG['model_name']}' inizializzato con successo sul disp

# Fai lo stesso test rapido di prima passando il batch reale per verificare che
try:
    # Usiamo le immagini estratte dal passo precedente (il batch del DataLoader
    test_inputs = images.to(CONFIG["device"])

    # Mettiamo il modello in modalità valutazione solo per il test rapido
    model.eval()
    with torch.no_grad():
        test_outputs = model(test_inputs)

    print("\n[VERIFICA DI SICUREZZA DELL'OUTPUT DEL NUOVO MODELLO]")
    print(f"    Shape dell'input inserito (Batch):  {test_inputs.shape}")
    print(f"    Shape dell'output ottenuto:           {test_outputs.shape}")
    print(f"    Significato: EfficientNet ha restituito correttamente {test_outp
    print("--> Struttura OK! Nessun errore di dimensionalità con il nuovo Backbo
except Exception as e:
    print(f"\n❌ Errore nel passaggio dei dati nel nuovo modello: {e}")

```

--> EfficientNet-B0 caricata con pesi pre-addestrati (ImageNet).

--> Testa di classificazione adattata: output impostato su 12 canali.

--> Modello 'efficientnet_b0' inizializzato con successo sul dispositivo: cuda

[VERIFICA DI SICUREZZA DELL'OUTPUT DEL NUOVO MODELLO]

Shape dell'input inserito (Batch): torch.Size([16, 3, 224, 224])

Shape dell'output ottenuto: torch.Size([16, 12])

Significato: EfficientNet ha restituito correttamente 16 predizioni, ciascuna composta da 12 logits.

--> Struttura OK! Nessun errore di dimensionalità con il nuovo Backbone.

CODICE PER GESTIRE SBILANCIAMENTO CLASSI

```

In [35]: import numpy as np
        from collections import Counter
        import torch

        # 1. Contiamo quante immagini ci sono per ogni sotto-classe nel TRAIN SET
        class_counts = Counter(train_labels)
        total_samples = len(train_labels)
        num_classes = 12

```

```
# 2. Calcoliamo i pesi invertendo la frequenza
weights = []
for cls_id in sorted(TRAIN_ID_TO_NAME.keys()):
    count = class_counts[cls_id]
    if count == 0: count = 1 # Protezione anti-crash se una classe avesse 0 camp
    # Formula standard: totale_campioni / (n_classi * campioni_classe)
    weight = total_samples / (num_classes * count)
    weights.append(weight)

# 3. Trasformiamo i pesi in un Tensor di PyTorch e lo spostiamo sulla GPU/CPU
class_weights = torch.FloatTensor(weights).to(CONFIG["device"])

print("🌀 PESI CALCOLATI PER IL BILANCIAMENTO DELLE CLASSI (12 CLASSI):")
print("-" * 65)
print(f"{'ID':<3} | {'Nome Sotto-Classe':<20} | {'Campioni':<10} | {'Peso Assegn'}")
print("-" * 65)

for cls_id in sorted(TRAIN_ID_TO_NAME.keys()):
    count = class_counts[cls_id]
    w = class_weights[cls_id].item() # Prende il valore numerico dal Tensor PyTo
    print(f"{'ID':<3} | {'TRAIN_ID_TO_NAME[cls_id]':<20} | {'count':<10} | {'w:.4f'}")
print("-" * 65)
```

🎯 PESI CALCOLATI PER IL BILANCIAMENTO DELLE CLASSI (12 CLASSI):

ID	Nome Sotto-Classe	Campioni	Peso Assegnato
0	battery	756	1.3682
1	clothes	4260	0.2428
2	shoes	1582	0.6538
3	brown	485	2.1326
4	green	503	2.0563
5	transparent	620	1.6683
6	metal	615	1.6818
7	organic	788	1.3126
8	cardboard	713	1.4507
9	paper	840	1.2313
10	plastic	692	1.4947
11	undifferentiated	558	1.8536

Questo blocco di codice implementa l'infrastruttura completa per l'addestramento e la validazione iterativa (Training Loop) del modello di baseline. Viene configurata la funzione di loss `CrossEntropyLoss` sulle 12 sotto-classi e l'ottimizzatore `Adam` per l'aggiornamento dei pesi della rete su dispositivo hardware. Ad ogni epoca, il ciclo estrae i logit del modello e monitora non solo le performance locali, ma calcola anche la `Balanced Accuracy` ufficiale sulle 8 macro-classi target, sfruttando il dizionario di conversione statico per garantire una valutazione coerente con i requisiti del report finale.

```
In [36]: import torch.optim as optim
          from sklearn.metrics import balanced_accuracy_score
          import torch
          import torch.nn as nn
          import numpy as np
          import time
```

=====

```

# 1. DEFINIZIONE DELLE FUNZIONI DI METRICA (Conversione da 12 a 8 classi)
# =====
def calculate_metrics(outputs_12, labels_12):
    """
    Prende le predizioni a 12 classi della rete e le label reali a 12 classi,
    calcola l'accuratezza standard a 12 classi E la Balanced Accuracy a 8 classi
    richiesta per la valutazione finale del progetto.
    """

    # Ottieni l'indice della sotto-classe con il punteggio (logit) più alto
    _, preds_12 = torch.max(outputs_12, 1)
    preds_12_np = preds_12.cpu().numpy()
    labels_12_np = labels_12.cpu().numpy()

    # Calcolo accuratezza standard sulle 12 sotto-classi (quante ne indovina su
    acc_12 = (preds_12_np == labels_12_np).mean()

    # CONVERSIONE SUL PONTE: Trasformiamo i vettori a 12 classi in vettori a 8 c
    preds_8_np = np.array([TRAIN_TO_TARGET_MAPPING[p] for p in preds_12_np])
    labels_8_np = np.array([TRAIN_TO_TARGET_MAPPING[l] for l in labels_12_np])

    # Calcolo della Balanced Accuracy ufficiale sulle 8 macro-classi
    balanced_acc_8 = balanced_accuracy_score(labels_8_np, preds_8_np)

    return acc_12, balanced_acc_8

# =====
# 2. CONFIGURAZIONE LOSS FUNCTION E OTTIMIZZATORE
# =====
criterion = nn.CrossEntropyLoss(weight=class_weights)
optimizer = optim.Adam(model.parameters(), lr=CONFIG["lr"], weight_decay=1e-4)

print("--> Funzioni di Loss e Ottimizzatore configurate correttamente.")
print(f"    Loss: CrossEntropyLoss (PESATA) 🔥 | Ottimizzatore: Adam (lr={CONFIG

# =====
# 3. CICLO DI ADDESTRAMENTO E VALIDAZIONE COMPLETO (Con Checkpoint del Best Mode
# =====
history = {"train_loss": [], "val_loss": [], "train_acc_12": [], "val_acc_12": [

# 🔥 Inizializzazione della variabile di controllo per salvare il modello miglio
best_val_loss = float('inf')
nome_file_checkpoint = 'best_model_weights.pth'

print(f"\n--> Avvio addestramento per {CONFIG['epochs']} epoche su dispositivo:
print("-" * 90)

for epoch in range(CONFIG["epochs"]):
    print(f"\n[DEBUG EPOCA {epoch+1}/{CONFIG['epochs']}] === INIZIO ===")
    t_inizio_epoca = time.time()

    # -----
    # FASE A: TRAINING (Addestramento sui dati di Train)
    # -----
    model.train()

    running_loss = 0.0
    epoch_outputs_12 = []
    epoch_labels_12 = []

```



```

# Inseriamo un flag per monitorare l'effettivo inizio del ciclo for
ciclo_train_iniziato = False

for batch_idx, (imgs, lbls_12, _) in enumerate(train_loader):
    if not ciclo_train_iniziato:
        print(f"[DEBUG TRAIN] SBLOCCATO! Il caricamento dal disco funziona.")
        ciclo_train_iniziato = True

    # Stampa ridotta: monitora solo ogni 50 batch per non intasare l'output
    if batch_idx % 50 == 0:
        print(f" -> [BATCH {batch_idx}/{len(train_loader)}] Spostamento dati")

    # Sposta i dati sulla GPU/CPU configurata
    imgs = imgs.to(CONFIG["device"])
    lbls_12 = lbls_12.to(CONFIG["device"])

    if batch_idx % 50 == 0:
        print(f" -> [BATCH {batch_idx}/{len(train_loader)}] Eseguo Forward")

    # Svuota i gradienti accumulati
    optimizer.zero_grad()

    # Forward pass
    outputs = model(imgs)

    # Calcolo della Loss
    loss = criterion(outputs, lbls_12)

    # Backward pass
    loss.backward()

    # Aggiornamento dei pesi
    optimizer.step()

    # Accumulo statistiche
    running_loss += loss.item() * imgs.size(0)
    epoch_outputs_12.append(outputs.detach())
    epoch_labels_12.append(lbls_12)

print(f"[DEBUG TRAIN] Fine ciclo for. Concateno i risultati dei {len(train_loader)} batch")
epoch_outputs_12 = torch.cat(epoch_outputs_12, dim=0)
epoch_labels_12 = torch.cat(epoch_labels_12, dim=0)

print(f"[DEBUG TRAIN] Calcolo metriche finali di Train per l'epoca...")
train_loss = running_loss / len(train_dataset)
train_acc_12, train_bal_8 = calculate_metrics(epoch_outputs_12, epoch_labels_12)

# -----
# FASE B: VALIDATION (Verifica sui dati di Validazione - No Gradienti)
# -----
model.eval()

val_running_loss = 0.0
val_outputs_12 = []
val_labels_12 = []

print(f"[DEBUG VAL] Accedo al 'val_loader'. Estraggo i batch...")
ciclo_val_iniziato = False

with torch.no_grad():

```

```

for batch_idx_val, (imgs, lbls_12, _) in enumerate(val_loader):
    if not ciclo_val_iniziato:
        print(f"[DEBUG VAL] SBLOCCATO! Elaboro il Batch {batch_idx_val}")
        ciclo_val_iniziato = True

    if batch_idx_val % 20 == 0:
        print(f" -> [VAL BATCH {batch_idx_val}/{len(val_loader)}] Calcolo")

    imgs = imgs.to(CONFIG["device"])
    lbls_12 = lbls_12.to(CONFIG["device"])

    outputs = model(imgs)
    loss = criterion(outputs, lbls_12)

    val_running_loss += loss.item() * imgs.size(0)
    val_outputs_12.append(outputs)
    val_labels_12.append(lbls_12)

print(f"[DEBUG VAL] Fine ciclo for. Concateno i risultati dei {len(val_loader)}")
val_outputs_12 = torch.cat(val_outputs_12, dim=0)
val_labels_12 = torch.cat(val_labels_12, dim=0)

print(f"[DEBUG VAL] Calcolo metriche finali di Validation...")
val_loss = val_running_loss / len(val_dataset)
val_acc_12, val_bal_8 = calculate_metrics(val_outputs_12, val_labels_12)

# Salva la cronologia dei dati nei vettori storici
history["train_loss"].append(train_loss)
history["val_loss"].append(val_loss)
history["train_acc_12"].append(train_acc_12)
history["val_acc_12"].append(val_acc_12)
history["train_bal_8"].append(train_bal_8)
history["val_bal_8"].append(val_bal_8)

t_fine_epoca = time.time()
print(f"[DEBUG] Epoca completata in {t_fine_epoca - t_inizio_epoca:.2f} secondi")

# -----
# STAMPA DEI PROGRESSI DI FINE EPOCA
# -----
print(f"==> Epoca [{epoch+1:02d}/{CONFIG['epochs']:02d}] | "
      f"Train Loss: {train_loss:.4f} - Val Loss: {val_loss:.4f} | "
      f"Acc 12cl (T/V): {train_acc_12*100:.1f}% / {val_acc_12*100:.1f}% | "
      f"B.Acc 8cl (T/V): {train_bal_8*100:.1f}% / {val_bal_8*100:.1f}%")

# -----
# 🔥 LOGICA DI CHECKPOINTING (Salvataggio del modello migliore)
# -----
if val_loss < best_val_loss:
    best_val_loss = val_loss # Aggiorna il minimo storico dell'errore

    # Snapshot dei parametri attuali e scrittura su disco
    torch.save(model.state_dict(), nome_file_checkpoint)
    print(f"    ✨ Modello ottimale trovato e blindato! Nuova minima Val Loss")
else:
    print(f"    ✖ Nessun miglioramento visibile. I pesi ottimali restano a")

print("-" * 90)
print("--> Addestramento della Baseline completato con successo!")

```

```
# =====  
# 4. RIPRISTINO AUTOMATICO DEI PESI MIGLIORI (Pre-Overfitting)  
# =====  
print(f"\n🔄 Ricarico i pesi ottimi registrati nel file '{nome_file_checkpoint}'")  
model.load_state_dict(torch.load(nome_file_checkpoint, map_location=CONFIG["device"]))  
print("--> [PRONTO] Il modello in memoria è stato ripristinato allo stato ottimale")
```

```

--> Funzioni di Loss e Ottimizzatore configurate correttamente.
    Loss: CrossEntropyLoss (PESATA) 🔥 | Ottimizzatore: Adam (lr=0.0001)

--> Avvio addestramento per 10 epoche su dispositivo: cuda...
-----
-----

[DEBUG EPOCA 1/10] === INIZIO ===
[DEBUG TRAIN] SBLOCCATO! Il caricamento dal disco funziona. Elaboro il Batch 0...
  -> [BATCH 0/776] Spostamento dati su cuda...
  -> [BATCH 0/776] Eseguo Forward + Backward pass...
  -> [BATCH 50/776] Spostamento dati su cuda...
  -> [BATCH 50/776] Eseguo Forward + Backward pass...
  -> [BATCH 100/776] Spostamento dati su cuda...
  -> [BATCH 100/776] Eseguo Forward + Backward pass...
  -> [BATCH 150/776] Spostamento dati su cuda...
  -> [BATCH 150/776] Eseguo Forward + Backward pass...
  -> [BATCH 200/776] Spostamento dati su cuda...
  -> [BATCH 200/776] Eseguo Forward + Backward pass...
  -> [BATCH 250/776] Spostamento dati su cuda...
  -> [BATCH 250/776] Eseguo Forward + Backward pass...
  -> [BATCH 300/776] Spostamento dati su cuda...
  -> [BATCH 300/776] Eseguo Forward + Backward pass...
  -> [BATCH 350/776] Spostamento dati su cuda...
  -> [BATCH 350/776] Eseguo Forward + Backward pass...
  -> [BATCH 400/776] Spostamento dati su cuda...
  -> [BATCH 400/776] Eseguo Forward + Backward pass...
  -> [BATCH 450/776] Spostamento dati su cuda...
  -> [BATCH 450/776] Eseguo Forward + Backward pass...
  -> [BATCH 500/776] Spostamento dati su cuda...
  -> [BATCH 500/776] Eseguo Forward + Backward pass...
  -> [BATCH 550/776] Spostamento dati su cuda...
  -> [BATCH 550/776] Eseguo Forward + Backward pass...
  -> [BATCH 600/776] Spostamento dati su cuda...
  -> [BATCH 600/776] Eseguo Forward + Backward pass...
  -> [BATCH 650/776] Spostamento dati su cuda...
  -> [BATCH 650/776] Eseguo Forward + Backward pass...
  -> [BATCH 700/776] Spostamento dati su cuda...
  -> [BATCH 700/776] Eseguo Forward + Backward pass...
  -> [BATCH 750/776] Spostamento dati su cuda...
  -> [BATCH 750/776] Eseguo Forward + Backward pass...
[DEBUG TRAIN] Fine ciclo for. Concateno i risultati dei 776 batch...
[DEBUG TRAIN] Calcolo metriche finali di Train per l'epoca...
[DEBUG VAL] Accedo al 'val_loader'. Estraggo i batch...
[DEBUG VAL] SBLOCCATO! Elaboro il Batch 0 di validazione...
  -> [VAL BATCH 0/194] Calcolo predizioni...
  -> [VAL BATCH 20/194] Calcolo predizioni...
  -> [VAL BATCH 40/194] Calcolo predizioni...
  -> [VAL BATCH 60/194] Calcolo predizioni...
  -> [VAL BATCH 80/194] Calcolo predizioni...
  -> [VAL BATCH 100/194] Calcolo predizioni...
  -> [VAL BATCH 120/194] Calcolo predizioni...
  -> [VAL BATCH 140/194] Calcolo predizioni...
  -> [VAL BATCH 160/194] Calcolo predizioni...
  -> [VAL BATCH 180/194] Calcolo predizioni...
[DEBUG VAL] Fine ciclo for. Concateno i risultati dei 194 batch...
[DEBUG VAL] Calcolo metriche finali di Validation...
[DEBUG] Epoca completata in 117.79 secondi.
==> Epoca [01/10] | Train Loss: 0.7816 - Val Loss: 0.2036 | Acc 12c1 (T/V): 82.0%
/ 95.3% | B.Acc 8c1 (T/V): 78.2% / 93.1%

```

🌟 Modello ottimale trovato e blindato! Nuova minima Val Loss: 0.2036

[DEBUG EPOCA 2/10] === INIZIO ===

[DEBUG TRAIN] SBLOCCATO! Il caricamento dal disco funziona. Elaboro il Batch 0...

```
-> [BATCH 0/776] Spostamento dati su cuda...
-> [BATCH 0/776] Eseguo Forward + Backward pass...
-> [BATCH 50/776] Spostamento dati su cuda...
-> [BATCH 50/776] Eseguo Forward + Backward pass...
-> [BATCH 100/776] Spostamento dati su cuda...
-> [BATCH 100/776] Eseguo Forward + Backward pass...
-> [BATCH 150/776] Spostamento dati su cuda...
-> [BATCH 150/776] Eseguo Forward + Backward pass...
-> [BATCH 200/776] Spostamento dati su cuda...
-> [BATCH 200/776] Eseguo Forward + Backward pass...
-> [BATCH 250/776] Spostamento dati su cuda...
-> [BATCH 250/776] Eseguo Forward + Backward pass...
-> [BATCH 300/776] Spostamento dati su cuda...
-> [BATCH 300/776] Eseguo Forward + Backward pass...
-> [BATCH 350/776] Spostamento dati su cuda...
-> [BATCH 350/776] Eseguo Forward + Backward pass...
-> [BATCH 400/776] Spostamento dati su cuda...
-> [BATCH 400/776] Eseguo Forward + Backward pass...
-> [BATCH 450/776] Spostamento dati su cuda...
-> [BATCH 450/776] Eseguo Forward + Backward pass...
-> [BATCH 500/776] Spostamento dati su cuda...
-> [BATCH 500/776] Eseguo Forward + Backward pass...
-> [BATCH 550/776] Spostamento dati su cuda...
-> [BATCH 550/776] Eseguo Forward + Backward pass...
-> [BATCH 600/776] Spostamento dati su cuda...
-> [BATCH 600/776] Eseguo Forward + Backward pass...
-> [BATCH 650/776] Spostamento dati su cuda...
-> [BATCH 650/776] Eseguo Forward + Backward pass...
-> [BATCH 700/776] Spostamento dati su cuda...
-> [BATCH 700/776] Eseguo Forward + Backward pass...
-> [BATCH 750/776] Spostamento dati su cuda...
-> [BATCH 750/776] Eseguo Forward + Backward pass...
```

[DEBUG TRAIN] Fine ciclo for. Concateno i risultati dei 776 batch...

[DEBUG TRAIN] Calcolo metriche finali di Train per l'epoca...

[DEBUG VAL] Accedo al 'val_loader'. Estraggo i batch...

[DEBUG VAL] SBLOCCATO! Elaboro il Batch 0 di validazione...

```
-> [VAL BATCH 0/194] Calcolo predizioni...
-> [VAL BATCH 20/194] Calcolo predizioni...
-> [VAL BATCH 40/194] Calcolo predizioni...
-> [VAL BATCH 60/194] Calcolo predizioni...
-> [VAL BATCH 80/194] Calcolo predizioni...
-> [VAL BATCH 100/194] Calcolo predizioni...
-> [VAL BATCH 120/194] Calcolo predizioni...
-> [VAL BATCH 140/194] Calcolo predizioni...
-> [VAL BATCH 160/194] Calcolo predizioni...
-> [VAL BATCH 180/194] Calcolo predizioni...
```

[DEBUG VAL] Fine ciclo for. Concateno i risultati dei 194 batch...

[DEBUG VAL] Calcolo metriche finali di Validation...

[DEBUG] Epoca completata in 156.80 secondi.

==> Epoca [02/10] | Train Loss: 0.2425 - Val Loss: 0.1287 | Acc 12cl (T/V): 93.7% / 96.4% | B.Acc 8cl (T/V): 93.0% / 95.9%

🌟 Modello ottimale trovato e blindato! Nuova minima Val Loss: 0.1287

[DEBUG EPOCA 3/10] === INIZIO ===

[DEBUG TRAIN] SBLOCCATO! Il caricamento dal disco funziona. Elaboro il Batch 0...

```
-> [BATCH 0/776] Spostamento dati su cuda...
```

```

-> [BATCH 0/776] Eseguo Forward + Backward pass...
-> [BATCH 50/776] Spostamento dati su cuda...
-> [BATCH 50/776] Eseguo Forward + Backward pass...
-> [BATCH 100/776] Spostamento dati su cuda...
-> [BATCH 100/776] Eseguo Forward + Backward pass...
-> [BATCH 150/776] Spostamento dati su cuda...
-> [BATCH 150/776] Eseguo Forward + Backward pass...
-> [BATCH 200/776] Spostamento dati su cuda...
-> [BATCH 200/776] Eseguo Forward + Backward pass...
-> [BATCH 250/776] Spostamento dati su cuda...
-> [BATCH 250/776] Eseguo Forward + Backward pass...
-> [BATCH 300/776] Spostamento dati su cuda...
-> [BATCH 300/776] Eseguo Forward + Backward pass...
-> [BATCH 350/776] Spostamento dati su cuda...
-> [BATCH 350/776] Eseguo Forward + Backward pass...
-> [BATCH 400/776] Spostamento dati su cuda...
-> [BATCH 400/776] Eseguo Forward + Backward pass...
-> [BATCH 450/776] Spostamento dati su cuda...
-> [BATCH 450/776] Eseguo Forward + Backward pass...
-> [BATCH 500/776] Spostamento dati su cuda...
-> [BATCH 500/776] Eseguo Forward + Backward pass...
-> [BATCH 550/776] Spostamento dati su cuda...
-> [BATCH 550/776] Eseguo Forward + Backward pass...
-> [BATCH 600/776] Spostamento dati su cuda...
-> [BATCH 600/776] Eseguo Forward + Backward pass...
-> [BATCH 650/776] Spostamento dati su cuda...
-> [BATCH 650/776] Eseguo Forward + Backward pass...
-> [BATCH 700/776] Spostamento dati su cuda...
-> [BATCH 700/776] Eseguo Forward + Backward pass...
-> [BATCH 750/776] Spostamento dati su cuda...
-> [BATCH 750/776] Eseguo Forward + Backward pass...
[DEBUG TRAIN] Fine ciclo for. Concateno i risultati dei 776 batch...
[DEBUG TRAIN] Calcolo metriche finali di Train per l'epoca...
[DEBUG VAL] Accedo al 'val_loader'. Estraggo i batch...
[DEBUG VAL] SBLOCCATO! Elaboro il Batch 0 di validazione...
  -> [VAL BATCH 0/194] Calcolo predizioni...
  -> [VAL BATCH 20/194] Calcolo predizioni...
  -> [VAL BATCH 40/194] Calcolo predizioni...
  -> [VAL BATCH 60/194] Calcolo predizioni...
  -> [VAL BATCH 80/194] Calcolo predizioni...
  -> [VAL BATCH 100/194] Calcolo predizioni...
  -> [VAL BATCH 120/194] Calcolo predizioni...
  -> [VAL BATCH 140/194] Calcolo predizioni...
  -> [VAL BATCH 160/194] Calcolo predizioni...
  -> [VAL BATCH 180/194] Calcolo predizioni...
[DEBUG VAL] Fine ciclo for. Concateno i risultati dei 194 batch...
[DEBUG VAL] Calcolo metriche finali di Validation...
[DEBUG] Epoca completata in 155.53 secondi.
==> Epoca [03/10] | Train Loss: 0.1491 - Val Loss: 0.1205 | Acc 12cl (T/V): 96.1%
/ 97.3% | B.Acc 8cl (T/V): 95.8% / 96.2%
  🌟 Modello ottimale trovato e blindato! Nuova minima Val Loss: 0.1205

[DEBUG EPOCA 4/10] === INIZIO ===
[DEBUG TRAIN] SBLOCCATO! Il caricamento dal disco funziona. Elaboro il Batch 0...
  -> [BATCH 0/776] Spostamento dati su cuda...
  -> [BATCH 0/776] Eseguo Forward + Backward pass...
  -> [BATCH 50/776] Spostamento dati su cuda...
  -> [BATCH 50/776] Eseguo Forward + Backward pass...
  -> [BATCH 100/776] Spostamento dati su cuda...
  -> [BATCH 100/776] Eseguo Forward + Backward pass...

```

```

-> [BATCH 150/776] Spostamento dati su cuda...
-> [BATCH 150/776] Eseguo Forward + Backward pass...
-> [BATCH 200/776] Spostamento dati su cuda...
-> [BATCH 200/776] Eseguo Forward + Backward pass...
-> [BATCH 250/776] Spostamento dati su cuda...
-> [BATCH 250/776] Eseguo Forward + Backward pass...
-> [BATCH 300/776] Spostamento dati su cuda...
-> [BATCH 300/776] Eseguo Forward + Backward pass...
-> [BATCH 350/776] Spostamento dati su cuda...
-> [BATCH 350/776] Eseguo Forward + Backward pass...
-> [BATCH 400/776] Spostamento dati su cuda...
-> [BATCH 400/776] Eseguo Forward + Backward pass...
-> [BATCH 450/776] Spostamento dati su cuda...
-> [BATCH 450/776] Eseguo Forward + Backward pass...
-> [BATCH 500/776] Spostamento dati su cuda...
-> [BATCH 500/776] Eseguo Forward + Backward pass...
-> [BATCH 550/776] Spostamento dati su cuda...
-> [BATCH 550/776] Eseguo Forward + Backward pass...
-> [BATCH 600/776] Spostamento dati su cuda...
-> [BATCH 600/776] Eseguo Forward + Backward pass...
-> [BATCH 650/776] Spostamento dati su cuda...
-> [BATCH 650/776] Eseguo Forward + Backward pass...
-> [BATCH 700/776] Spostamento dati su cuda...
-> [BATCH 700/776] Eseguo Forward + Backward pass...
-> [BATCH 750/776] Spostamento dati su cuda...
-> [BATCH 750/776] Eseguo Forward + Backward pass...
[DEBUG TRAIN] Fine ciclo for. Concateno i risultati dei 776 batch...
[DEBUG TRAIN] Calcolo metriche finali di Train per l'epoca...
[DEBUG VAL] Accedo al 'val_loader'. Estraggo i batch...
[DEBUG VAL] SBLOCCATO! Elaboro il Batch 0 di validazione...
  -> [VAL BATCH 0/194] Calcolo predizioni...
  -> [VAL BATCH 20/194] Calcolo predizioni...
  -> [VAL BATCH 40/194] Calcolo predizioni...
  -> [VAL BATCH 60/194] Calcolo predizioni...
  -> [VAL BATCH 80/194] Calcolo predizioni...
  -> [VAL BATCH 100/194] Calcolo predizioni...
  -> [VAL BATCH 120/194] Calcolo predizioni...
  -> [VAL BATCH 140/194] Calcolo predizioni...
  -> [VAL BATCH 160/194] Calcolo predizioni...
  -> [VAL BATCH 180/194] Calcolo predizioni...
[DEBUG VAL] Fine ciclo for. Concateno i risultati dei 194 batch...
[DEBUG VAL] Calcolo metriche finali di Validation...
[DEBUG] Epoca completata in 169.74 secondi.
==> Epoca [04/10] | Train Loss: 0.0939 - Val Loss: 0.1120 | Acc 12cl (T/V): 97.4%
/ 97.6% | B.Acc 8cl (T/V): 97.3% / 97.1%
  🌟 Modello ottimale trovato e blindato! Nuova minima Val Loss: 0.1120

```

```

[DEBUG EPOCA 5/10] === INIZIO ===
[DEBUG TRAIN] SBLOCCATO! Il caricamento dal disco funziona. Elaboro il Batch 0...
  -> [BATCH 0/776] Spostamento dati su cuda...
  -> [BATCH 0/776] Eseguo Forward + Backward pass...
  -> [BATCH 50/776] Spostamento dati su cuda...
  -> [BATCH 50/776] Eseguo Forward + Backward pass...
  -> [BATCH 100/776] Spostamento dati su cuda...
  -> [BATCH 100/776] Eseguo Forward + Backward pass...
  -> [BATCH 150/776] Spostamento dati su cuda...
  -> [BATCH 150/776] Eseguo Forward + Backward pass...
  -> [BATCH 200/776] Spostamento dati su cuda...
  -> [BATCH 200/776] Eseguo Forward + Backward pass...
  -> [BATCH 250/776] Spostamento dati su cuda...

```



```

-> [BATCH 250/776] Eseguo Forward + Backward pass...
-> [BATCH 300/776] Spostamento dati su cuda...
-> [BATCH 300/776] Eseguo Forward + Backward pass...
-> [BATCH 350/776] Spostamento dati su cuda...
-> [BATCH 350/776] Eseguo Forward + Backward pass...
-> [BATCH 400/776] Spostamento dati su cuda...
-> [BATCH 400/776] Eseguo Forward + Backward pass...
-> [BATCH 450/776] Spostamento dati su cuda...
-> [BATCH 450/776] Eseguo Forward + Backward pass...
-> [BATCH 500/776] Spostamento dati su cuda...
-> [BATCH 500/776] Eseguo Forward + Backward pass...
-> [BATCH 550/776] Spostamento dati su cuda...
-> [BATCH 550/776] Eseguo Forward + Backward pass...
-> [BATCH 600/776] Spostamento dati su cuda...
-> [BATCH 600/776] Eseguo Forward + Backward pass...
-> [BATCH 650/776] Spostamento dati su cuda...
-> [BATCH 650/776] Eseguo Forward + Backward pass...
-> [BATCH 700/776] Spostamento dati su cuda...
-> [BATCH 700/776] Eseguo Forward + Backward pass...
-> [BATCH 750/776] Spostamento dati su cuda...
-> [BATCH 750/776] Eseguo Forward + Backward pass...
[DEBUG TRAIN] Fine ciclo for. Concateno i risultati dei 776 batch...
[DEBUG TRAIN] Calcolo metriche finali di Train per l'epoca...
[DEBUG VAL] Accedo al 'val_loader'. Estraggo i batch...
[DEBUG VAL] SBLOCCATO! Elaboro il Batch 0 di validazione...
  -> [VAL BATCH 0/194] Calcolo predizioni...
  -> [VAL BATCH 20/194] Calcolo predizioni...
  -> [VAL BATCH 40/194] Calcolo predizioni...
  -> [VAL BATCH 60/194] Calcolo predizioni...
  -> [VAL BATCH 80/194] Calcolo predizioni...
  -> [VAL BATCH 100/194] Calcolo predizioni...
  -> [VAL BATCH 120/194] Calcolo predizioni...
  -> [VAL BATCH 140/194] Calcolo predizioni...
  -> [VAL BATCH 160/194] Calcolo predizioni...
  -> [VAL BATCH 180/194] Calcolo predizioni...
[DEBUG VAL] Fine ciclo for. Concateno i risultati dei 194 batch...
[DEBUG VAL] Calcolo metriche finali di Validation...
[DEBUG] Epoca completata in 162.68 secondi.
==> Epoca [05/10] | Train Loss: 0.0713 - Val Loss: 0.1236 | Acc 12cl (T/V): 97.9%
/ 97.4% | B.Acc 8cl (T/V): 97.8% / 96.4%
  ✗ Nessun miglioramento visibile. I pesi ottimali restano ancorati alla Val
  Loss di: 0.1120

```

```

[DEBUG EPOCA 6/10] === INIZIO ===
[DEBUG TRAIN] SBLOCCATO! Il caricamento dal disco funziona. Elaboro il Batch 0...
  -> [BATCH 0/776] Spostamento dati su cuda...
  -> [BATCH 0/776] Eseguo Forward + Backward pass...
  -> [BATCH 50/776] Spostamento dati su cuda...
  -> [BATCH 50/776] Eseguo Forward + Backward pass...
  -> [BATCH 100/776] Spostamento dati su cuda...
  -> [BATCH 100/776] Eseguo Forward + Backward pass...
  -> [BATCH 150/776] Spostamento dati su cuda...
  -> [BATCH 150/776] Eseguo Forward + Backward pass...
  -> [BATCH 200/776] Spostamento dati su cuda...
  -> [BATCH 200/776] Eseguo Forward + Backward pass...
  -> [BATCH 250/776] Spostamento dati su cuda...
  -> [BATCH 250/776] Eseguo Forward + Backward pass...
  -> [BATCH 300/776] Spostamento dati su cuda...
  -> [BATCH 300/776] Eseguo Forward + Backward pass...
  -> [BATCH 350/776] Spostamento dati su cuda...

```



```

-> [BATCH 350/776] Eseguo Forward + Backward pass...
-> [BATCH 400/776] Spostamento dati su cuda...
-> [BATCH 400/776] Eseguo Forward + Backward pass...
-> [BATCH 450/776] Spostamento dati su cuda...
-> [BATCH 450/776] Eseguo Forward + Backward pass...
-> [BATCH 500/776] Spostamento dati su cuda...
-> [BATCH 500/776] Eseguo Forward + Backward pass...
-> [BATCH 550/776] Spostamento dati su cuda...
-> [BATCH 550/776] Eseguo Forward + Backward pass...
-> [BATCH 600/776] Spostamento dati su cuda...
-> [BATCH 600/776] Eseguo Forward + Backward pass...
-> [BATCH 650/776] Spostamento dati su cuda...
-> [BATCH 650/776] Eseguo Forward + Backward pass...
-> [BATCH 700/776] Spostamento dati su cuda...
-> [BATCH 700/776] Eseguo Forward + Backward pass...
-> [BATCH 750/776] Spostamento dati su cuda...
-> [BATCH 750/776] Eseguo Forward + Backward pass...
[DEBUG TRAIN] Fine ciclo for. Concateno i risultati dei 776 batch...
[DEBUG TRAIN] Calcolo metriche finali di Train per l'epoca...
[DEBUG VAL] Accedo al 'val_loader'. Estraggo i batch...
[DEBUG VAL] SBLOCCATO! Elaboro il Batch 0 di validazione...
  -> [VAL BATCH 0/194] Calcolo predizioni...
  -> [VAL BATCH 20/194] Calcolo predizioni...
  -> [VAL BATCH 40/194] Calcolo predizioni...
  -> [VAL BATCH 60/194] Calcolo predizioni...
  -> [VAL BATCH 80/194] Calcolo predizioni...
  -> [VAL BATCH 100/194] Calcolo predizioni...
  -> [VAL BATCH 120/194] Calcolo predizioni...
  -> [VAL BATCH 140/194] Calcolo predizioni...
  -> [VAL BATCH 160/194] Calcolo predizioni...
  -> [VAL BATCH 180/194] Calcolo predizioni...
[DEBUG VAL] Fine ciclo for. Concateno i risultati dei 194 batch...
[DEBUG VAL] Calcolo metriche finali di Validation...
[DEBUG] Epoca completata in 169.78 secondi.
==> Epoca [06/10] | Train Loss: 0.0597 - Val Loss: 0.1222 | Acc 12cl (T/V): 98.5%
/ 97.4% | B.Acc 8cl (T/V): 98.4% / 96.5%
  ✖ Nessun miglioramento visibile. I pesi ottimali restano ancorati alla Val
  Loss di: 0.1120

```

```

[DEBUG EPOCA 7/10] === INIZIO ===
[DEBUG TRAIN] SBLOCCATO! Il caricamento dal disco funziona. Elaboro il Batch 0...
  -> [BATCH 0/776] Spostamento dati su cuda...
  -> [BATCH 0/776] Eseguo Forward + Backward pass...
  -> [BATCH 50/776] Spostamento dati su cuda...
  -> [BATCH 50/776] Eseguo Forward + Backward pass...
  -> [BATCH 100/776] Spostamento dati su cuda...
  -> [BATCH 100/776] Eseguo Forward + Backward pass...
  -> [BATCH 150/776] Spostamento dati su cuda...
  -> [BATCH 150/776] Eseguo Forward + Backward pass...
  -> [BATCH 200/776] Spostamento dati su cuda...
  -> [BATCH 200/776] Eseguo Forward + Backward pass...
  -> [BATCH 250/776] Spostamento dati su cuda...
  -> [BATCH 250/776] Eseguo Forward + Backward pass...
  -> [BATCH 300/776] Spostamento dati su cuda...
  -> [BATCH 300/776] Eseguo Forward + Backward pass...
  -> [BATCH 350/776] Spostamento dati su cuda...
  -> [BATCH 350/776] Eseguo Forward + Backward pass...
  -> [BATCH 400/776] Spostamento dati su cuda...
  -> [BATCH 400/776] Eseguo Forward + Backward pass...
  -> [BATCH 450/776] Spostamento dati su cuda...

```

```
-> [BATCH 450/776] Eseguo Forward + Backward pass...
-> [BATCH 500/776] Spostamento dati su cuda...
-> [BATCH 500/776] Eseguo Forward + Backward pass...
-> [BATCH 550/776] Spostamento dati su cuda...
-> [BATCH 550/776] Eseguo Forward + Backward pass...
-> [BATCH 600/776] Spostamento dati su cuda...
-> [BATCH 600/776] Eseguo Forward + Backward pass...
-> [BATCH 650/776] Spostamento dati su cuda...
-> [BATCH 650/776] Eseguo Forward + Backward pass...
-> [BATCH 700/776] Spostamento dati su cuda...
-> [BATCH 700/776] Eseguo Forward + Backward pass...
-> [BATCH 750/776] Spostamento dati su cuda...
-> [BATCH 750/776] Eseguo Forward + Backward pass...
[DEBUG TRAIN] Fine ciclo for. Concateno i risultati dei 776 batch...
[DEBUG TRAIN] Calcolo metriche finali di Train per l'epoca...
[DEBUG VAL] Accedo al 'val_loader'. Estraggo i batch...
[DEBUG VAL] SBLOCCATO! Elaboro il Batch 0 di validazione...
-> [VAL BATCH 0/194] Calcolo predizioni...
-> [VAL BATCH 20/194] Calcolo predizioni...
-> [VAL BATCH 40/194] Calcolo predizioni...
-> [VAL BATCH 60/194] Calcolo predizioni...
-> [VAL BATCH 80/194] Calcolo predizioni...
-> [VAL BATCH 100/194] Calcolo predizioni...
-> [VAL BATCH 120/194] Calcolo predizioni...
-> [VAL BATCH 140/194] Calcolo predizioni...
-> [VAL BATCH 160/194] Calcolo predizioni...
-> [VAL BATCH 180/194] Calcolo predizioni...
[DEBUG VAL] Fine ciclo for. Concateno i risultati dei 194 batch...
[DEBUG VAL] Calcolo metriche finali di Validation...
[DEBUG] Epoca completata in 148.89 secondi.
==> Epoca [07/10] | Train Loss: 0.0412 - Val Loss: 0.1188 | Acc 12cl (T/V): 98.8%
/ 97.5% | B.Acc 8cl (T/V): 99.0% / 97.1%
✗ Nessun miglioramento visibile. I pesi ottimali restano ancorati alla Val
Loss di: 0.1120
```

```
[DEBUG EPOCA 8/10] === INIZIO ===
[DEBUG TRAIN] SBLOCCATO! Il caricamento dal disco funziona. Elaboro il Batch 0...
-> [BATCH 0/776] Spostamento dati su cuda...
-> [BATCH 0/776] Eseguo Forward + Backward pass...
-> [BATCH 50/776] Spostamento dati su cuda...
-> [BATCH 50/776] Eseguo Forward + Backward pass...
-> [BATCH 100/776] Spostamento dati su cuda...
-> [BATCH 100/776] Eseguo Forward + Backward pass...
-> [BATCH 150/776] Spostamento dati su cuda...
-> [BATCH 150/776] Eseguo Forward + Backward pass...
-> [BATCH 200/776] Spostamento dati su cuda...
-> [BATCH 200/776] Eseguo Forward + Backward pass...
-> [BATCH 250/776] Spostamento dati su cuda...
-> [BATCH 250/776] Eseguo Forward + Backward pass...
-> [BATCH 300/776] Spostamento dati su cuda...
-> [BATCH 300/776] Eseguo Forward + Backward pass...
-> [BATCH 350/776] Spostamento dati su cuda...
-> [BATCH 350/776] Eseguo Forward + Backward pass...
-> [BATCH 400/776] Spostamento dati su cuda...
-> [BATCH 400/776] Eseguo Forward + Backward pass...
-> [BATCH 450/776] Spostamento dati su cuda...
-> [BATCH 450/776] Eseguo Forward + Backward pass...
-> [BATCH 500/776] Spostamento dati su cuda...
-> [BATCH 500/776] Eseguo Forward + Backward pass...
-> [BATCH 550/776] Spostamento dati su cuda...
```

```

-> [BATCH 550/776] Eseguo Forward + Backward pass...
-> [BATCH 600/776] Spostamento dati su cuda...
-> [BATCH 600/776] Eseguo Forward + Backward pass...
-> [BATCH 650/776] Spostamento dati su cuda...
-> [BATCH 650/776] Eseguo Forward + Backward pass...
-> [BATCH 700/776] Spostamento dati su cuda...
-> [BATCH 700/776] Eseguo Forward + Backward pass...
-> [BATCH 750/776] Spostamento dati su cuda...
-> [BATCH 750/776] Eseguo Forward + Backward pass...
[DEBUG TRAIN] Fine ciclo for. Concateno i risultati dei 776 batch...
[DEBUG TRAIN] Calcolo metriche finali di Train per l'epoca...
[DEBUG VAL] Accedo al 'val_loader'. Estraggo i batch...
[DEBUG VAL] SBLOCCATO! Elaboro il Batch 0 di validazione...
  -> [VAL BATCH 0/194] Calcolo predizioni...
  -> [VAL BATCH 20/194] Calcolo predizioni...
  -> [VAL BATCH 40/194] Calcolo predizioni...
  -> [VAL BATCH 60/194] Calcolo predizioni...
  -> [VAL BATCH 80/194] Calcolo predizioni...
  -> [VAL BATCH 100/194] Calcolo predizioni...
  -> [VAL BATCH 120/194] Calcolo predizioni...
  -> [VAL BATCH 140/194] Calcolo predizioni...
  -> [VAL BATCH 160/194] Calcolo predizioni...
  -> [VAL BATCH 180/194] Calcolo predizioni...
[DEBUG VAL] Fine ciclo for. Concateno i risultati dei 194 batch...
[DEBUG VAL] Calcolo metriche finali di Validation...
[DEBUG] Epoca completata in 147.08 secondi.
==> Epoca [08/10] | Train Loss: 0.0403 - Val Loss: 0.1420 | Acc 12cl (T/V): 98.8%
/ 97.0% | B.Acc 8cl (T/V): 98.9% / 96.0%
  ✗ Nessun miglioramento visibile. I pesi ottimali restano ancorati alla Val
  Loss di: 0.1120

```

```

[DEBUG EPOCA 9/10] === INIZIO ===
[DEBUG TRAIN] SBLOCCATO! Il caricamento dal disco funziona. Elaboro il Batch 0...
  -> [BATCH 0/776] Spostamento dati su cuda...
  -> [BATCH 0/776] Eseguo Forward + Backward pass...
  -> [BATCH 50/776] Spostamento dati su cuda...
  -> [BATCH 50/776] Eseguo Forward + Backward pass...
  -> [BATCH 100/776] Spostamento dati su cuda...
  -> [BATCH 100/776] Eseguo Forward + Backward pass...
  -> [BATCH 150/776] Spostamento dati su cuda...
  -> [BATCH 150/776] Eseguo Forward + Backward pass...
  -> [BATCH 200/776] Spostamento dati su cuda...
  -> [BATCH 200/776] Eseguo Forward + Backward pass...
  -> [BATCH 250/776] Spostamento dati su cuda...
  -> [BATCH 250/776] Eseguo Forward + Backward pass...
  -> [BATCH 300/776] Spostamento dati su cuda...
  -> [BATCH 300/776] Eseguo Forward + Backward pass...
  -> [BATCH 350/776] Spostamento dati su cuda...
  -> [BATCH 350/776] Eseguo Forward + Backward pass...
  -> [BATCH 400/776] Spostamento dati su cuda...
  -> [BATCH 400/776] Eseguo Forward + Backward pass...
  -> [BATCH 450/776] Spostamento dati su cuda...
  -> [BATCH 450/776] Eseguo Forward + Backward pass...
  -> [BATCH 500/776] Spostamento dati su cuda...
  -> [BATCH 500/776] Eseguo Forward + Backward pass...
  -> [BATCH 550/776] Spostamento dati su cuda...
  -> [BATCH 550/776] Eseguo Forward + Backward pass...
  -> [BATCH 600/776] Spostamento dati su cuda...
  -> [BATCH 600/776] Eseguo Forward + Backward pass...
  -> [BATCH 650/776] Spostamento dati su cuda...

```

```

-> [BATCH 650/776] Eseguo Forward + Backward pass...
-> [BATCH 700/776] Spostamento dati su cuda...
-> [BATCH 700/776] Eseguo Forward + Backward pass...
-> [BATCH 750/776] Spostamento dati su cuda...
-> [BATCH 750/776] Eseguo Forward + Backward pass...
[DEBUG TRAIN] Fine ciclo for. Concateno i risultati dei 776 batch...
[DEBUG TRAIN] Calcolo metriche finali di Train per l'epoca...
[DEBUG VAL] Accedo al 'val_loader'. Estraggo i batch...
[DEBUG VAL] SBLOCCATO! Elaboro il Batch 0 di validazione...
  -> [VAL BATCH 0/194] Calcolo predizioni...
  -> [VAL BATCH 20/194] Calcolo predizioni...
  -> [VAL BATCH 40/194] Calcolo predizioni...
  -> [VAL BATCH 60/194] Calcolo predizioni...
  -> [VAL BATCH 80/194] Calcolo predizioni...
  -> [VAL BATCH 100/194] Calcolo predizioni...
  -> [VAL BATCH 120/194] Calcolo predizioni...
  -> [VAL BATCH 140/194] Calcolo predizioni...
  -> [VAL BATCH 160/194] Calcolo predizioni...
  -> [VAL BATCH 180/194] Calcolo predizioni...
[DEBUG VAL] Fine ciclo for. Concateno i risultati dei 194 batch...
[DEBUG VAL] Calcolo metriche finali di Validation...
[DEBUG] Epoca completata in 143.47 secondi.
==> Epoca [09/10] | Train Loss: 0.0414 - Val Loss: 0.1278 | Acc 12cl (T/V): 98.8%
/ 97.5% | B.Acc 8cl (T/V): 98.9% / 96.7%
  ✗ Nessun miglioramento visibile. I pesi ottimali restano ancorati alla Val
  Loss di: 0.1120

```

```

[DEBUG EPOCA 10/10] === INIZIO ===
[DEBUG TRAIN] SBLOCCATO! Il caricamento dal disco funziona. Elaboro il Batch 0...
  -> [BATCH 0/776] Spostamento dati su cuda...
  -> [BATCH 0/776] Eseguo Forward + Backward pass...
  -> [BATCH 50/776] Spostamento dati su cuda...
  -> [BATCH 50/776] Eseguo Forward + Backward pass...
  -> [BATCH 100/776] Spostamento dati su cuda...
  -> [BATCH 100/776] Eseguo Forward + Backward pass...
  -> [BATCH 150/776] Spostamento dati su cuda...
  -> [BATCH 150/776] Eseguo Forward + Backward pass...
  -> [BATCH 200/776] Spostamento dati su cuda...
  -> [BATCH 200/776] Eseguo Forward + Backward pass...
  -> [BATCH 250/776] Spostamento dati su cuda...
  -> [BATCH 250/776] Eseguo Forward + Backward pass...
  -> [BATCH 300/776] Spostamento dati su cuda...
  -> [BATCH 300/776] Eseguo Forward + Backward pass...
  -> [BATCH 350/776] Spostamento dati su cuda...
  -> [BATCH 350/776] Eseguo Forward + Backward pass...
  -> [BATCH 400/776] Spostamento dati su cuda...
  -> [BATCH 400/776] Eseguo Forward + Backward pass...
  -> [BATCH 450/776] Spostamento dati su cuda...
  -> [BATCH 450/776] Eseguo Forward + Backward pass...
  -> [BATCH 500/776] Spostamento dati su cuda...
  -> [BATCH 500/776] Eseguo Forward + Backward pass...
  -> [BATCH 550/776] Spostamento dati su cuda...
  -> [BATCH 550/776] Eseguo Forward + Backward pass...
  -> [BATCH 600/776] Spostamento dati su cuda...
  -> [BATCH 600/776] Eseguo Forward + Backward pass...
  -> [BATCH 650/776] Spostamento dati su cuda...
  -> [BATCH 650/776] Eseguo Forward + Backward pass...
  -> [BATCH 700/776] Spostamento dati su cuda...
  -> [BATCH 700/776] Eseguo Forward + Backward pass...
  -> [BATCH 750/776] Spostamento dati su cuda...

```

```
-> [BATCH 750/776] Eseguo Forward + Backward pass...
[DEBUG TRAIN] Fine ciclo for. Concateno i risultati dei 776 batch...
[DEBUG TRAIN] Calcolo metriche finali di Train per l'epoca...
[DEBUG VAL] Accedo al 'val_loader'. Estraggo i batch...
[DEBUG VAL] SBLOCCATO! Elaboro il Batch 0 di validazione...
-> [VAL BATCH 0/194] Calcolo predizioni...
-> [VAL BATCH 20/194] Calcolo predizioni...
-> [VAL BATCH 40/194] Calcolo predizioni...
-> [VAL BATCH 60/194] Calcolo predizioni...
-> [VAL BATCH 80/194] Calcolo predizioni...
-> [VAL BATCH 100/194] Calcolo predizioni...
-> [VAL BATCH 120/194] Calcolo predizioni...
-> [VAL BATCH 140/194] Calcolo predizioni...
-> [VAL BATCH 160/194] Calcolo predizioni...
-> [VAL BATCH 180/194] Calcolo predizioni...
[DEBUG VAL] Fine ciclo for. Concateno i risultati dei 194 batch...
[DEBUG VAL] Calcolo metriche finali di Validation...
[DEBUG] Epoca completata in 134.57 secondi.
==> Epoca [10/10] | Train Loss: 0.0371 - Val Loss: 0.1261 | Acc 12cl (T/V): 98.9%
/ 97.3% | B.Acc 8cl (T/V): 99.0% / 96.5%
  ✖ Nessun miglioramento visibile. I pesi ottimali restano ancorati alla Val
  Loss di: 0.1120
-----
-----
--> Addestramento della Baseline completato con successo!

🔄 Ricarico i pesi ottimi registrati nel file 'best_model_weights.pth' per neutr
alizzare l'overfitting...
```

```

RuntimeError                                Traceback (most recent call last)
Cell In[36], line 194
    190 # =====
    191 # 4. RIPRISTINO AUTOMATICO DEI PESI MIGLIORI (Pre-Overfitting)
    192 # =====
    193 print(f"\n🔄 Ricarico i pesi ottimi registrati nel file '{nome_file_checkpoint}' per neutralizzare l'overfitting...")
--> 194 model.load_state_dict(torch.load(nome_file_checkpoint, map_location=CONFIG["device"]))
    195 print("--> [PRONTO] Il modello in memoria è stato ripristinato allo stato ottimale ed è pronto per i test di massa!")

File /usr/local/lib/python3.10/dist-packages/torch/nn/modules/module.py:2593, in Module.load_state_dict(self, state_dict, strict, assign)
    2585         error_msgs.insert(
    2586             0,
    2587             "Missing key(s) in state_dict: {}".format(
    2588                 ", ".join(f'"{k}"' for k in missing_keys)
    2589             ),
    2590         )
    2592 if len(error_msgs) > 0:
-> 2593     raise RuntimeError(
    2594         "Error(s) in loading state_dict for {}: \n\t{}".format(
    2595             self.__class__.__name__, "\n\t".join(error_msgs)
    2596         )
    2597     )
    2598 return _IncompatibleKeys(missing_keys, unexpected_keys)

RuntimeError: Error(s) in loading state_dict for WasteNet:
Unexpected key(s) in state_dict: "backbone.features.1.1.block.0.0.weight", "backbone.features.1.1.block.0.1.weight", "backbone.features.1.1.block.0.1.bias", "backbone.features.1.1.block.0.1.running_mean", "backbone.features.1.1.block.0.1.running_var", "backbone.features.1.1.block.0.1.num_batches_tracked", "backbone.features.1.1.block.1.fc1.weight", "backbone.features.1.1.block.1.fc1.bias", "backbone.features.1.1.block.1.fc2.weight", "backbone.features.1.1.block.1.fc2.bias", "backbone.features.1.1.block.2.0.weight", "backbone.features.1.1.block.2.1.weight", "backbone.features.1.1.block.2.1.bias", "backbone.features.1.1.block.2.1.running_mean", "backbone.features.1.1.block.2.1.running_var", "backbone.features.1.1.block.2.1.num_batches_tracked", "backbone.features.2.2.block.0.0.weight", "backbone.features.2.2.block.0.1.weight", "backbone.features.2.2.block.0.1.bias", "backbone.features.2.2.block.0.1.running_mean", "backbone.features.2.2.block.0.1.running_var", "backbone.features.2.2.block.0.1.num_batches_tracked", "backbone.features.2.2.block.1.0.weight", "backbone.features.2.2.block.1.1.weight", "backbone.features.2.2.block.1.1.bias", "backbone.features.2.2.block.1.1.running_mean", "backbone.features.2.2.block.1.1.running_var", "backbone.features.2.2.block.1.1.num_batches_tracked", "backbone.features.2.2.block.2.fc1.weight", "backbone.features.2.2.block.2.fc1.bias", "backbone.features.2.2.block.2.fc2.weight", "backbone.features.2.2.block.2.fc2.bias", "backbone.features.2.2.block.3.0.weight", "backbone.features.2.2.block.3.1.weight", "backbone.features.2.2.block.3.1.bias", "backbone.features.2.2.block.3.1.running_mean", "backbone.features.2.2.block.3.1.running_var", "backbone.features.2.2.block.3.1.num_batches_tracked", "backbone.features.3.2.block.0.0.weight", "backbone.features.3.2.block.0.1.weight", "backbone.features.3.2.block.0.1.bias", "backbone.features.3.2.block.0.1.running_mean", "backbone.features.3.2.block.0.1.running_var", "backbone.features.3.2.block.0.1.num_batches_tracked", "backbone.features.3.2.block.1.0.weight", "backbone.features.3.2.block.1.1.weight", "backbone.features.3.2.block.1.1.bias", "backbone.features.3.2.block.1.1.running_mean", "backbone.features.3.2.block.1.1.running_var", "backbone.features.3.2.block.1.1.num_batches_tracked", "backbone.features.3.2.block.2.fc1.weight", "backbone.features.3.2.block.2.fc1.bias", "backbone.features.3.2.block.2.fc2.weight", "backbone.features.3.2.block.2.fc2.bias"

```



```

2.weight", "backbone.features.3.2.block.2.fc2.bias", "backbone.features.3.2.block.3.0.weight", "backbone.features.3.2.block.3.1.weight", "backbone.features.3.2.block.3.1.bias", "backbone.features.3.2.block.3.1.running_mean", "backbone.features.3.2.block.3.1.running_var", "backbone.features.3.2.block.3.1.num_batches_tracked", "backbone.features.4.3.block.0.0.weight", "backbone.features.4.3.block.0.1.weight", "backbone.features.4.3.block.0.1.bias", "backbone.features.4.3.block.0.1.running_mean", "backbone.features.4.3.block.0.1.running_var", "backbone.features.4.3.block.0.1.num_batches_tracked", "backbone.features.4.3.block.1.0.weight", "backbone.features.4.3.block.1.1.weight", "backbone.features.4.3.block.1.1.bias", "backbone.features.4.3.block.1.1.running_mean", "backbone.features.4.3.block.1.1.running_var", "backbone.features.4.3.block.1.1.num_batches_tracked", "backbone.features.4.3.block.2.fc1.weight", "backbone.features.4.3.block.2.fc1.bias", "backbone.features.4.3.block.2.fc2.weight", "backbone.features.4.3.block.2.fc2.bias", "backbone.features.4.3.block.3.0.weight", "backbone.features.4.3.block.3.1.weight", "backbone.features.4.3.block.3.1.bias", "backbone.features.4.3.block.3.1.running_mean", "backbone.features.4.3.block.3.1.running_var", "backbone.features.4.3.block.3.1.num_batches_tracked", "backbone.features.5.3.block.0.0.weight", "backbone.features.5.3.block.0.1.weight", "backbone.features.5.3.block.0.1.bias", "backbone.features.5.3.block.0.1.running_mean", "backbone.features.5.3.block.0.1.running_var", "backbone.features.5.3.block.0.1.num_batches_tracked", "backbone.features.5.3.block.1.0.weight", "backbone.features.5.3.block.1.1.weight", "backbone.features.5.3.block.1.1.bias", "backbone.features.5.3.block.1.1.running_mean", "backbone.features.5.3.block.1.1.running_var", "backbone.features.5.3.block.1.1.num_batches_tracked", "backbone.features.5.3.block.2.fc1.weight", "backbone.features.5.3.block.2.fc1.bias", "backbone.features.5.3.block.2.fc2.weight", "backbone.features.5.3.block.2.fc2.bias", "backbone.features.5.3.block.3.0.weight", "backbone.features.5.3.block.3.1.weight", "backbone.features.5.3.block.3.1.bias", "backbone.features.5.3.block.3.1.running_mean", "backbone.features.5.3.block.3.1.running_var", "backbone.features.5.3.block.3.1.num_batches_tracked", "backbone.features.6.4.block.0.0.weight", "backbone.features.6.4.block.0.1.weight", "backbone.features.6.4.block.0.1.bias", "backbone.features.6.4.block.0.1.running_mean", "backbone.features.6.4.block.0.1.running_var", "backbone.features.6.4.block.0.1.num_batches_tracked", "backbone.features.6.4.block.1.0.weight", "backbone.features.6.4.block.1.1.weight", "backbone.features.6.4.block.1.1.bias", "backbone.features.6.4.block.1.1.running_mean", "backbone.features.6.4.block.1.1.running_var", "backbone.features.6.4.block.1.1.num_batches_tracked", "backbone.features.6.4.block.2.fc1.weight", "backbone.features.6.4.block.2.fc1.bias", "backbone.features.6.4.block.2.fc2.weight", "backbone.features.6.4.block.2.fc2.bias", "backbone.features.6.4.block.3.0.weight", "backbone.features.6.4.block.3.1.weight", "backbone.features.6.4.block.3.1.bias", "backbone.features.6.4.block.3.1.running_mean", "backbone.features.6.4.block.3.1.running_var", "backbone.features.6.4.block.3.1.num_batches_tracked", "backbone.features.7.1.block.0.0.weight", "backbone.features.7.1.block.0.1.weight", "backbone.features.7.1.block.0.1.bias", "backbone.features.7.1.block.0.1.running_mean", "backbone.features.7.1.block.0.1.running_var", "backbone.features.7.1.block.0.1.num_batches_tracked", "backbone.features.7.1.block.1.0.weight", "backbone.features.7.1.block.1.1.weight", "backbone.features.7.1.block.1.1.bias", "backbone.features.7.1.block.1.1.running_mean", "backbone.features.7.1.block.1.1.running_var", "backbone.features.7.1.block.1.1.num_batches_tracked", "backbone.features.7.1.block.2.fc1.weight", "backbone.features.7.1.block.2.fc1.bias", "backbone.features.7.1.block.2.fc2.weight", "backbone.features.7.1.block.2.fc2.bias", "backbone.features.7.1.block.3.0.weight", "backbone.features.7.1.block.3.1.weight", "backbone.features.7.1.block.3.1.bias", "backbone.features.7.1.block.3.1.running_mean", "backbone.features.7.1.block.3.1.running_var", "backbone.features.7.1.block.3.1.num_batches_tracked".

```

```

In [9]: import matplotlib.pyplot as plt
import numpy as np

def visualize_model_predictions(model, val_loader, id_to_name, num_images=8):
    model.eval() # Modalità valutazione: spegne il Dropout per fare predizioni

```

```

# Prendiamo un batch dal validation set
images, labels_12, labels_8 = next(iter(val_loader))

# Mandiamo le immagini alla GPU/CPU
inputs = images.to(CONFIG["device"])

with torch.no_grad():
    outputs = model(inputs)
    _, preds = torch.max(outputs, 1)

# =====
# STAMPA DI VERIFICA (DEBUG SATURAZIONE CLASSE)
# =====
print("\n🔍 --- ANALISI DELLE PREDIZIONI SUL BATCH DI VALIDATION ---")
lista_predizioni = [id_to_name[p.item()] for p in preds]
lista_reali = [id_to_name[l.item()] for l in labels_12]

print(f"Predizioni effettuate dalla rete su questo batch ({len(preds)} immagini)")
print(lista_predizioni)

# Contiamo quante volte ha predetto ogni classe per beccare il bug visivamente
from collections import Counter
conteggio = Counter(lista_predizioni)
print("\nConteggio delle classi predette:")
for classe, qta in conteggio.items():
    print(f"    └─ {classe}: predetto {qta} volte su {len(preds)}")
print("-----\n")
# =====

# Configura la griglia (2 righe e 4 colonne per 8 immagini)
cols = 4
rows = int(np.ceil(num_images / cols))
fig = plt.figure(figsize=(16, rows * 4))

for i in range(num_images):
    ax = fig.add_subplot(rows, cols, i + 1)

    # Denormalizziamo l'immagine per mostrare i colori reali
    img = images[i].permute(1, 2, 0).cpu().numpy()
    mean = np.array([0.485, 0.456, 0.406])
    std = np.array([0.229, 0.224, 0.225])
    img = std * img + mean
    img = np.clip(img, 0, 1)

    plt.imshow(img)

    actual_name = id_to_name[labels_12[i].item()]
    pred_name = id_to_name[preds[i].item()]

    # Verde se indovinato, Rosso se sbagliato
    color = 'green' if preds[i] == labels_12[i] else 'red'

    ax.set_title(f"Reale: {actual_name}\nPred: {pred_name}", color=color, fontweight='bold')
    ax.axis('off')

plt.tight_layout()
plt.show()

```



```
# Lanciamo la funzione (Assicurati che val_loader e TRAIN_ID_TO_NAME esistano)
visualize_model_predictions(model, val_loader, id_to_name=TRAIN_ID_TO_NAME, num_
```

```
-----
NameError                                Traceback (most recent call last)
Cell In[9], line 66
      63 plt.show()
      65 # Lanciamo la funzione (Assicurati che val_loader e TRAIN_ID_TO_NAME esis
tano)
--> 66 visualize_model_predictions(model, val_loader, id_to_name=TRAIN_ID_TO_NAM
E, num_images=8)

NameError: name 'model' is not defined
```

```
In [37]: import torch
from collections import Counter
import pandas as pd

def analyze_all_predictions_summary(model, val_loader, id_to_name):
    """
    Scorre l'intero validation set per calcolare quante volte il modello
    ha predetto ciascuna classe rispetto alla distribuzione reale,
    senza mostrare alcun grafico.
    """
    model.eval() # Modalità valutazione stabile

    tutte_le_predizioni = []
    tutte_le_label_reali = []
    totale_immagini = 0

    print("🔄 Elaborazione di tutti i batch del Validation Set in corso...")

    with torch.no_grad():
        for imgs, lbls_12, _ in val_loader:
            imgs = imgs.to(CONFIG["device"])

            # Predizione del modello
            outputs = model(imgs)
            _, preds = torch.max(outputs, 1)

            # Accumuliamo i nomi testuali delle classi
            tutte_le_predizioni.extend([id_to_name[p.item()] for p in preds])
            tutte_le_label_reali.extend([id_to_name[l.item()] for l in lbls_12])
            totale_immagini += imgs.size(0)

    # Conteggio delle frequenze
    conteggio_predette = Counter(tutte_le_predizioni)
    conteggio_reali = Counter(tutte_le_label_reali)

    # Prendiamo la lista unica di tutte le classi incontrate per fare un confron
    tutte_le_classi = sorted(list(set(tutte_le_predizioni + tutte_le_label_reali

    # =====
    # STAMPA DEL REPORT STATISTICO
    # =====
    print("\n" + "="*80)
    print(f"📊 REPORT GLOBALE DELLE PREDIZIONI (Analisi su {totale_immagini} im
    print("="*80)
    print(f"{'CLASSE (Sotto-classe 12c)':<30} | {'PREDETTE (Q.tà)':<18} | {'REAL
    print("-"*80)
```

```

for classe in tutte_le_classi:
    qta_predetta = conteggio_predette[classe]
    qta_reale = conteggio_reali[classe]

    # Calcoliamo anche una percentuale indicativa per il prof
    pct_predetta = (qta_predetta / totale_immagini) * 100
    pct_reale = (qta_reale / totale_immagini) * 100

    str_predetta = f"{qta_predetta} ({pct_predetta:.1f}%)"
    str_reale = f"{qta_reale} ({pct_reale:.1f}%)"

    print(f"{classe:<30} | {str_predetta:<18} | {str_reale:<15}")

print("="*80)

# Un piccolo check di controllo per l'overfitting/saturazione
classe_piu_predetta, max_predizioni = conteggio_predette.most_common(1)[0]
pct_max = (max_predizioni / totale_immagini) * 100
print(f"💡 Info bilanciamento: La classe più assegnata dal modello è '{classe_piu_predetta}' ({max_predizioni} volte, pari al {pct_max:.1f}% del totale).")
print("="*80 + "\n")

# Lanciamo l'analisi globale su tutto il validation set
analyze_all_predictions_summary(model, val_loader, id_to_name=TRAIN_ID_TO_NAME)

```

🔄 Elaborazione di tutti i batch del Validation Set in corso...

```

=====
📊 REPORT GLOBALE DELLE PREDIZIONI (Analisi su 3103 immagini totali)
=====

```

CLASSE (Sotto-classe 12c)	PREDETTE (Q.tà)	REALI (Q.tà)
battery	198 (6.4%)	189 (6.1%)
brown	132 (4.3%)	122 (3.9%)
cardboard	175 (5.6%)	178 (5.7%)
clothes	1060 (34.2%)	1065 (34.3%)
green	130 (4.2%)	126 (4.1%)
metal	154 (5.0%)	154 (5.0%)
organic	210 (6.8%)	197 (6.3%)
paper	195 (6.3%)	210 (6.8%)
plastic	163 (5.3%)	173 (5.6%)
shoes	396 (12.8%)	395 (12.7%)
transparent	156 (5.0%)	155 (5.0%)
undifferentiated	134 (4.3%)	139 (4.5%)

```

=====
💡 Info bilanciamento: La classe più assegnata dal modello è 'clothes' (1060 volte, pari al 34.2% del totale).
=====

```

6. Loss Function e Meccanismo di Unione Finale delle Classi La funzione di conversione mappa l'output generato a 12 classi direttamente nelle 8 macro-classi richieste dalla valutazione, calcolando i pesi della Cross-Entropy per compensare lo sbilanciamento del dataset. Magari possiamo iniziare con la loss normale di Cross Entropy, e poi fare quella per compensare le classi meno numerose

7. Matrice di confusione alla fine della validazione

```

In [38]: import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import torch
from sklearn.metrics import confusion_matrix

# =====
# 1. ESTRAZIONE RAPIDA PREDIZIONI SUL VALIDATION SET (SENZA RIADDESTRARE!)
# =====
print("🔍 Estrazione rapida delle predizioni dal Validation Set...")
model.eval()

all_preds_8 = []
all_labels_8 = []

with torch.no_grad():
    for imgs, lbls_12, _ in val_loader:
        imgs = imgs.to(CONFIG["device"])

        outputs = model(imgs)
        _, preds_12 = torch.max(outputs, 1)

        # Mappatura istantanea da 12 sotto-classi a 8 macro-classi
        preds_8 = [TRAIN_TO_TARGET_MAPPING[p.item()] for p in preds_12]
        labels_8 = [TRAIN_TO_TARGET_MAPPING[l.item()] for l in lbls_12]

        all_preds_8.extend(preds_8)
        all_labels_8.extend(labels_8)

final_val_preds_8 = np.array(all_preds_8)
final_val_labels_8 = np.array(all_labels_8)
print("✅ Estrazione completata! Generazione grafici in corso...")
print("-" * 100)

# =====
# 2. CALCOLO DELLA MATRICE DI CONFUSIONE (Assoluta e Normalizzata)
# =====
macro_classes_names = [TARGET_ID_TO_NAME[i] for i in sorted(TARGET_ID_TO_NAME.keys())]

cm_absolute = confusion_matrix(final_val_labels_8, final_val_preds_8)
cm_normalized = cm_absolute.astype('float') / cm_absolute.sum(axis=1)[:, np.newaxis]

# =====
# 3. CREAZIONE DEL PLOT GRAFICO AFFIANCATO (CORRETTO PER MATPLOTLIB)
# =====
fig, axes = plt.subplots(1, 2, figsize=(22, 9))

# --- MAPPA A: Conteggi Assoluti ---
sns.heatmap(
    cm_absolute,
    annot=True,
    fmt="d",
    cmap="Blues", # Scritto minuscolo/standard
    xticklabels=macro_classes_names,
    yticklabels=macro_classes_names,
    square=True,

```

```

cbar=True,
ax=axes[0]
)
axes[0].set_title("A. Conteggi Assoluti (Numero Immagini)", fontsize=14, fontwei
axes[0].set_ylabel("Classe Reale (Ground Truth)", fontsize=12, fontweight='bold'
axes[0].set_xlabel("Classe Predetta dal Modello", fontsize=12, fontweight='bold'
axes[0].set_xticklabels(macro_classes_names, rotation=45, ha='right')

# --- MAPPA B: Percentuali Normalizzate ---
annot_labels = np.array(["{0:.1f}%".format(value*100) for value in row] for row

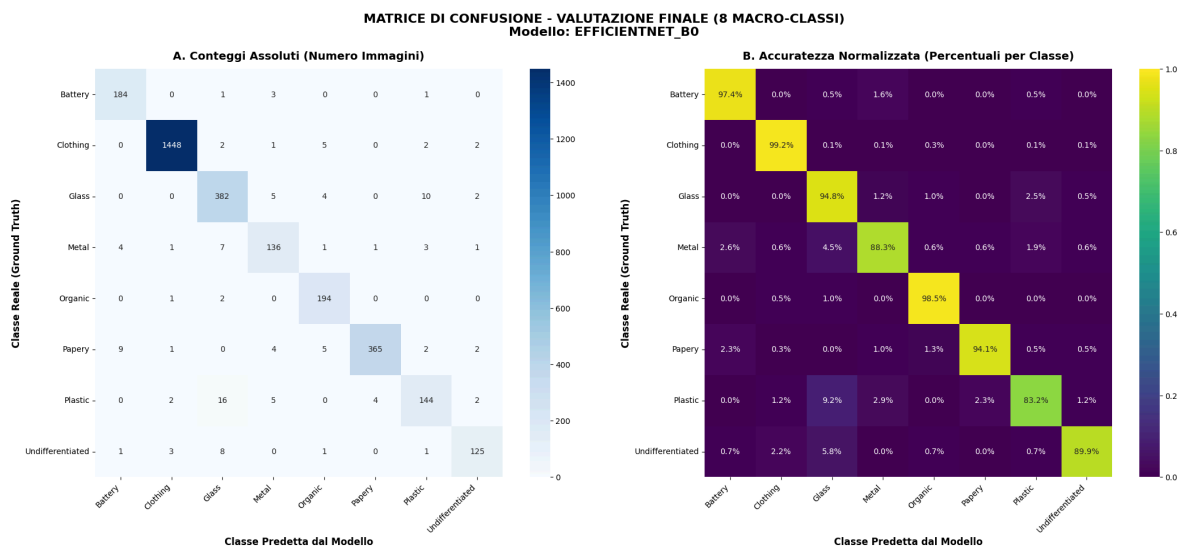
sns.heatmap(
    cm_normalized,
    annot=annot_labels,
    fmt="",
    cmap="viridis", # 🔥 CORREZIONE: 'viridis' tutto in minuscolo, accettato un
    xticklabels=macro_classes_names,
    yticklabels=macro_classes_names,
    square=True,
    vmax=1.0,
    vmin=0.0,
    cbar=True,
    ax=axes[1]
)
axes[1].set_title("B. Accuratezza Normalizzata (Percentuali per Classe)", fontsi
axes[1].set_ylabel("Classe Reale (Ground Truth)", fontsize=12, fontweight='bold'
axes[1].set_xlabel("Classe Predetta dal Modello", fontsize=12, fontweight='bold'
axes[1].set_xticklabels(macro_classes_names, rotation=45, ha='right')

# Titolo superiore e salvataggio
plt.suptitle(f"MATRICE DI CONFUSIONE - VALUTAZIONE FINALE (8 MACRO-CLASSI)\nMode
plt.tight_layout()
plt.savefig("matrice_confusione_finale.png", dpi=300, bbox_inches='tight')
plt.show()

```

🔗 Estrazione rapida delle predizioni dal Validation Set...

✅ Estrazione completata! Generazione grafici in corso...



8. Loop di Training e Validazione (con Balanced Accuracy) Questo ciclo addestra lo strato lineare finale del modello e calcola la metrica richiesta (Balanced Accuracy) sulle 8 macro-classi. -- fatta
9. Grafico per trade-off tra training error e validation error? Dopo 5-10 epoche, calcolo la differenza tra le curve. grafico del learning rate, registrare le curve di accuratezza e sovrapporre grafico trade-off accuratezza e velocità

```
In [39]: import matplotlib.pyplot as plt
import numpy as np

# 🔥 CORREZIONE DEFINITIVA: Rimosse le parentesi da plt.style.available
if 'seaborn-v0_8-whitegrid' in plt.style.available:
    plt.style.use('seaborn-v0_8-whitegrid')
else:
    plt.style.use('default')

epoche = np.arange(1, len(history["train_loss"]) + 1)

# =====
# GRAFICO 1: TRADE-OFF TRA TRAINING LOSS E VALIDATION LOSS (ERRORE)
# =====
plt.figure(figsize=(14, 5))

plt.subplot(1, 2, 1)
plt.plot(epoche, history["train_loss"], 'o-', label='Training Loss (Error)', col
plt.plot(epoche, history["val_loss"], 's-', label='Validation Loss (Error)', col

# Calcolo del punto di minimo o evidenza della forbice dopo 5-10 epoche
gap_epoca_10 = history["val_loss"][9] - history["train_loss"][9] if len(epoche)
if gap_epoca_10 is not None:
    plt.axvline(x=10, color='red', linestyle='--', alpha=0.7)
    plt.text(10.5, history["val_loss"][9], f'Gap Overfitting\nall\'Epoca 10: {ga

plt.title("Trade-off: Training Error vs Validation Error", fontsize=12, fontweig
plt.xlabel("Epoca", fontsize=10)
plt.ylabel("Loss (Errore)", fontsize=10)
plt.xticks(epoche)
plt.legend(fontsize=10)

# =====
# GRAFICO 2: CURVE DI ACCURATEZZA SOVRAPPOSTE (12 Classi vs 8 Classi)
# =====
plt.subplot(1, 2, 2)
# Trasformiamo in percentuali per una lettura più accademica
plt.plot(epoche, [a*100 for a in history["train_acc_12"]], ':', label='Train Acc
plt.plot(epoche, [a*100 for a in history["val_acc_12"]], '-', label='Val Acc (12
plt.plot(epoche, [a*100 for a in history["train_bal_8"]], '--', label='Train Bal
plt.plot(epoche, [a*100 for a in history["val_bal_8"]], 'd-', label='Val Balance

plt.title("Sovrapposizione Curve di Accuratezza (12cl vs 8cl Target)", fontsize=
plt.xlabel("Epoca", fontsize=10)
plt.ylabel("Accuratezza (%)", fontsize=10)
plt.xticks(epoche)
plt.ylim(0, 105)
plt.legend(fontsize=9, loc='lower right')
```

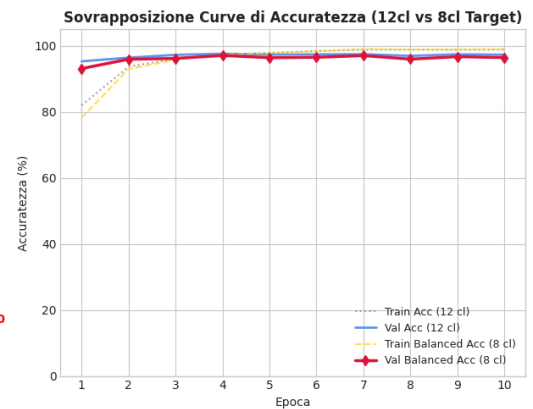
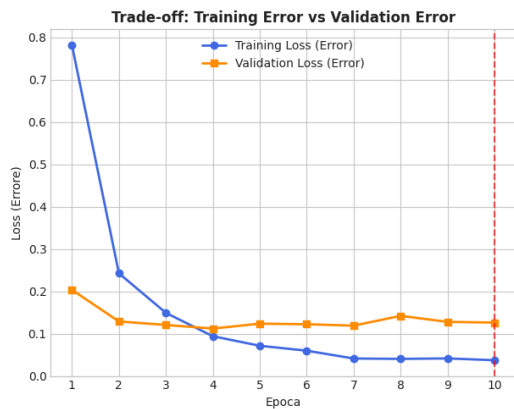
```
plt.tight_layout()
plt.show()

# =====
# ANALISI DI TRADE-OFF: ACCURATEZZA VS VELOCITÀ (Testuale per il Prof)
# =====
print("\n" + "="*80)
print("📊 ANALISI PRESTAZIONALE PER LA RELAZIONE (TRADE-OFF ACCURATEZZA/VELOCITÀ)")
print("="*80)

# FIX DI SICUREZZA: Controlla se tempo_impiegato esiste nella memoria locale, a
tempo_medio_epoca = locals().get('tempo_impiegato', 0.0)

if tempo_medio_epoca > 0:
    print(f"🕒 Velocità computazionale media: {tempo_medio_epoca:.2f} secondi |")
else:
    print(f"🕒 Velocità computazionale registrata su dispositivo: {CONFIG['dev:

print(f"📈 Massima Balanced Accuracy (8 macro-classi) raggiunta: {max(history['
print(f"📉 Minimo Errore di Validazione (Val Loss) registrato: {min(history['val
print(f"🚀 Configurazione iperparametri utilizzata: LR={CONFIG['lr']} | Batch S
print("="*80)
```



=====

📊 ANALISI PRESTAZIONALE PER LA RELAZIONE (TRADE-OFF ACCURATEZZA/VELOCITÀ)

=====

🕒 Velocità computazionale registrata su dispositivo: cuda.

📈 Massima Balanced Accuracy (8 macro-classi) raggiunta: 97.10%

📉 Minimo Errore di Validazione (Val Loss) registrato: 0.1120

🚀 Configurazione iperparametri utilizzata: LR=0.0001 | Batch Size=16 | Modello=efficientnet_b0

=====

```
In [45]: import io
import zipfile
from pathlib import Path
import torch
from PIL import Image

# =====
# 1. FUNZIONE DI INFERENZA DA ZIP (Sfrutta il tuo modello già in memoria)
# =====
def predict_image_from_zip(zip_file, internal_path, model, config):
    """Esegue l'inferenza leggendo i byte direttamente dallo zip senza salvare s
    img_bytes = zip_file.read(internal_path)

    with Image.open(io.BytesIO(img_bytes)) as img:
```

```

        img_rgb = img.convert("RGB")
        # Sfrutta la tua funzione globale delle trasformazioni
        inference_transform = get_transforms_pipeline(config, is_training_set=False)
        input_tensor = inference_transform(img_rgb).unsqueeze(0)

input_tensor = input_tensor.to(config["device"])

# Passaggio sul tuo modello esistente
model.eval()
with torch.no_grad():
    outputs_12 = model(input_tensor)
    probs_12 = torch.softmax(outputs_12, dim=1).squeeze(0).cpu()

# Mappatura vettorizzata sulle 8 macro-classi target
mapping_tensor = torch.tensor(
    [TRAIN_TO_TARGET_MAPPING[i] for i in range(len(probs_12))], dtype=torch.float
)
probs_8 = torch.zeros(config["num_classes_target"])
probs_8.index_add_(0, mapping_tensor, probs_12)

pred_8_id = int(torch.argmax(probs_8).item())
return pred_8_id

# =====
# 2. CICLO DI TEST DI MASSA SULL'ARCHIVIO ZIP (Aggiornato)
# =====
def run_mass_test_on_zip(zip_path, model, config):
    zip_path = Path(zip_path)
    if not zip_path.exists():
        raise FileNotFoundError(f"Archivio non trovato: {zip_path.resolve()}")

    print("\n" + "=" * 60)
    print(f"🚀 AVVIO TEST DI MASSA SUL DATASET: {zip_path.name}")
    print("=" * 60)

    estensioni_valide = {".jpg", ".jpeg", ".png", ".bmp"}
    conteggio_classi = {}
    immagini_da_processare = []

    # Creiamo al volo una mappa lowercase per non fallire se le cartelle sono mi
    # Usiamo TARGET_LABEL_MAP che è già definito nel tuo secondo blocco
    target_map_lowercase = {k.lower(): v for k, v in TARGET_LABEL_MAP.items()}

    # Esplorazione rapida dello ZIP per mappare la struttura delle cartelle
    with zipfile.ZipFile(zip_path, "r") as z:
        for file_info in z.infolist():
            if file_info.is_dir() or file_info.filename.startswith("__MACOSX"):
                continue

            path_interno = Path(file_info.filename)
            if path_interno.suffix.lower() in estensioni_valide:
                # Trasformiamo in minuscolo il nome della cartella per il confronto
                classe_reale_nome = path_interno.parent.name.lower()

                # Controllo sulla mappa corretta
                if classe_reale_nome in target_map_lowercase:
                    classe_reale_id = target_map_lowercase[classe_reale_nome]
                    immagini_da_processare.append((file_info.filename, classe_reale_id))
                    conteggio_classi[classe_reale_nome] = conteggio_classi.get(classe_reale_nome, 0) + 1

```



```

totale_foto = len(immagini_da_processare)
if totale_foto == 0:
    print("⚠ Nessuna immagine trovata. Verifica che i nomi delle cartelle c
    return

print(f"📁 Trovate {totale_foto} immagini totali divise in {len(conteggio_c
for k, v in conteggio_classi.items():
    print(f"    📦 {k:<20}: {v} foto")

predizioni_corrette = 0
tutte_predizioni = []
tutti_target_reali = []

print(f"\n💡 Elaborazione di massa in corso su {config['device']}...")

# Avvio del ciclo effettivo senza alterare lo stato della rete
with zipfile.ZipFile(zip_path, "r") as z:
    for idx, (filename, label_reale) in enumerate(immagini_da_processare, st
        pred_id = predict_image_from_zip(z, filename, model, config)

        tutte_predizioni.append(pred_id)
        tutti_target_reali.append(label_reale)

        if pred_id == label_reale:
            predizioni_corrette += 1

    # Stampa un feedback ogni 100 immagini per monitorare la velocità
    if idx % 100 == 0 or idx == totale_foto:
        acc_parziale = (predizioni_corrette / idx) * 100
        print(f"    [Progresso: {idx}/{totale_foto}] Accuratezza corrent

accuratezza_finale = (predizioni_corrette / totale_foto) * 100

print("\n" + "=" * 60)
print("📊 REPORT FINALE DI VERIFICA SULLO ZIP")
print("=" * 60)
print(f"Immagini totali elaborate:         {totale_foto}")
print(f"Predizioni corrette (Esatte):        {predizioni_corrette}")
print(f"Accuratezza Globale (Accuracy): {accuratezza_finale:.2f}%")
print("=" * 60 + "\n")

return tutti_target_reali, tutte_predizioni

# =====
# 3. CHIAMATA DI TEST (Usa le variabili già attive nel tuo notebook)
# =====
archivio_target = "archive.zip"

# Avvia il test usando direttamente 'model' e 'CONFIG' presenti nella tua memori
risultati_test = run_mass_test_on_zip(
    zip_path=archivio_target,
    model=model,          # Prende la rete corrente senza resettarla o riaddestrarla
    config=CONFIG          # Prende il tuo dizionario globale CONFIG
)

```

```
=====
🚀 AVVIO TEST DI MASSA SUL DATASET: archive.zip
=====
```

```
📁 Trovate 2786 immagini totali divise in 3 classi target.
├── 📁 glass : 1002 foto
├── 📁 metal : 820 foto
└── 📁 plastic : 964 foto
```

```
💡 Elaborazione di massa in corso su cuda...
[Progresso: 100/2786] Accuratezza corrente: 88.00%
[Progresso: 200/2786] Accuratezza corrente: 83.50%
[Progresso: 300/2786] Accuratezza corrente: 86.33%
[Progresso: 400/2786] Accuratezza corrente: 84.50%
[Progresso: 500/2786] Accuratezza corrente: 86.00%
[Progresso: 600/2786] Accuratezza corrente: 86.50%
[Progresso: 700/2786] Accuratezza corrente: 87.43%
[Progresso: 800/2786] Accuratezza corrente: 88.12%
[Progresso: 900/2786] Accuratezza corrente: 88.44%
[Progresso: 1000/2786] Accuratezza corrente: 88.30%
[Progresso: 1100/2786] Accuratezza corrente: 88.36%
[Progresso: 1200/2786] Accuratezza corrente: 88.67%
[Progresso: 1300/2786] Accuratezza corrente: 88.77%
[Progresso: 1400/2786] Accuratezza corrente: 88.14%
[Progresso: 1500/2786] Accuratezza corrente: 87.87%
[Progresso: 1600/2786] Accuratezza corrente: 87.50%
[Progresso: 1700/2786] Accuratezza corrente: 87.76%
[Progresso: 1800/2786] Accuratezza corrente: 87.33%
[Progresso: 1900/2786] Accuratezza corrente: 87.47%
[Progresso: 2000/2786] Accuratezza corrente: 87.60%
[Progresso: 2100/2786] Accuratezza corrente: 87.90%
[Progresso: 2200/2786] Accuratezza corrente: 88.05%
[Progresso: 2300/2786] Accuratezza corrente: 88.17%
[Progresso: 2400/2786] Accuratezza corrente: 88.21%
[Progresso: 2500/2786] Accuratezza corrente: 88.24%
[Progresso: 2600/2786] Accuratezza corrente: 88.31%
[Progresso: 2700/2786] Accuratezza corrente: 88.33%
[Progresso: 2786/2786] Accuratezza corrente: 88.08%
```

```
=====
📊 REPORT FINALE DI VERIFICA SULLO ZIP
=====
```

```
Immagini totali elaborate:      2786
Predizioni corrette (Esatte):   2454
Accuratezza Globale (Accuracy): 88.08%
=====
```

```
In [48]: import io
import zipfile
from pathlib import Path
import torch
from PIL import Image

# =====
# 1. FUNZIONE DI INFERENZA DA ZIP (Invariata)
# =====
def predict_image_from_zip(zip_file, internal_path, model, config):
    """Esegue l'inferenza leggendo i byte direttamente dallo zip senza salvare s
    img_bytes = zip_file.read(internal_path)
```

```

with Image.open(io.BytesIO(img_bytes)) as img:
    img_rgb = img.convert("RGB")
    inference_transform = get_transforms_pipeline(config, is_training_set=False)
    input_tensor = inference_transform(img_rgb).unsqueeze(0)

input_tensor = input_tensor.to(config["device"])

model.eval()
with torch.no_grad():
    outputs_12 = model(input_tensor)
    probs_12 = torch.softmax(outputs_12, dim=1).squeeze(0).cpu()

mapping_tensor = torch.tensor(
    [TRAIN_TO_TARGET_MAPPING[i] for i in range(len(probs_12))], dtype=torch.
)
probs_8 = torch.zeros(config["num_classes_target"])
probs_8.index_add_(0, mapping_tensor, probs_12)

pred_8_id = int(torch.argmax(probs_8).item())
return pred_8_id

# =====
# 2. CICLO DI TEST DI MASSA SULL'ARCHIVIO ZIP (Corretto per Paper/Cardboard)
# =====
def run_mass_test_on_zip(zip_path, model, config):
    zip_path = Path(zip_path)
    if not zip_path.exists():
        raise FileNotFoundError(f"Archivio non trovato: {zip_path.resolve()}")

    print("\n" + "=" * 60)
    print(f"🚀 AVVIO TEST DI MASSA SUL DATASET: {zip_path.name}")
    print("=" * 60)

    estensioni_valide = {".jpg", ".jpeg", ".png", ".bmp"}
    conteggio_classi = {}
    immagini_da_processare = []

    # Creiamo la mappa base in minuscolo
    target_map_lowercase = {k.lower(): v for k, v in TARGET_LABEL_MAP.items()}

    # AGGIUNTA FONDAMENTALE: Indiziamo esplicitamente "paper" e "cardboard" ver
    id_papery = TARGET_LABEL_MAP["Papery"]
    target_map_lowercase["paper"] = id_papery
    target_map_lowercase["cardboard"] = id_papery

    # Esplorazione rapida dello ZIP
    with zipfile.ZipFile(zip_path, "r") as z:
        for file_info in z.infolist():
            if file_info.is_dir() or file_info.filename.startswith("__MACOSX") or
                continue

            path_interno = Path(file_info.filename)
            if path_interno.suffix.lower() in estensioni_valide:
                classe_reale_nome = path_interno.parent.name.lower()

                # Ora "paper" e "cardboard" verranno riconosciuti e associati al
                if classe_reale_nome in target_map_lowercase:
                    classe_reale_id = target_map_lowercase[classe_reale_nome]
                    immagini_da_processare.append((file_info.filename, classe_re

```

```

        conteggio_classi[classe_reale_nome] = conteggio_classi.get(c

totale_foto = len(immagini_da_processare)
if totale_foto == 0:
    print("⚠️ Nessuna immagine trovata. Verifica i nomi delle cartelle dentro la cartella")
    return

print(f"📁 Trovate {totale_foto} immagini totali divise in {len(conteggio_classi)} categorie")
for k, v in conteggio_classi.items():
    print(f"📁 {k:<20}: {v} foto")

predizioni_corrette = 0
tutte_predizioni = []
tutti_target_reali = []

print(f"\n🚀 Elaborazione di massa in corso su {config['device']}...")

with zipfile.ZipFile(zip_path, "r") as z:
    for idx, (filename, label_reale) in enumerate(immagini_da_processare, start=1):
        pred_id = predict_image_from_zip(z, filename, model, config)

        tutte_predizioni.append(pred_id)
        tutti_target_reali.append(label_reale)

        if pred_id == label_reale:
            predizioni_corrette += 1

        if idx % 100 == 0 or idx == totale_foto:
            acc_parziale = (predizioni_corrette / idx) * 100
            print(f"📊 [Progresso: {idx}/{totale_foto}] Accuratezza corrente: {acc_parziale:.2f}%")

accuratezza_finale = (predizioni_corrette / totale_foto) * 100

print("\n" + "=" * 60)
print("📊 REPORT FINALE DI VERIFICA SULLO ZIP")
print("=" * 60)
print(f"Immagini totali elaborate: {totale_foto}")
print(f"Predizioni corrette (Esatte): {predizioni_corrette}")
print(f"Accuratezza Globale (Accuracy): {accuratezza_finale:.2f}%")
print("=" * 60 + "\n")

return tutti_target_reali, tutte_predizioni

# =====
# 3. CHIAMATA DI TEST
# =====
archivio_target = "archive.zip"

risultati_test = run_mass_test_on_zip(
    zip_path=archivio_target,
    model=model,
    config=CONFIG
)

```

=====

🚀 AVVIO TEST DI MASSA SUL DATASET: archive.zip

=====

📁 Trovate 4780 immagini totali divise in 5 sotto-cartelle nello ZIP.

└─ 📦 cardboard	: 806 foto
└─ 📦 glass	: 1002 foto
└─ 📦 metal	: 820 foto
└─ 📦 paper	: 1188 foto
└─ 📦 plastic	: 964 foto

💡 Elaborazione di massa in corso su cuda...

[Progresso: 100/4780]	Accuratezza corrente: 97.00%
[Progresso: 200/4780]	Accuratezza corrente: 97.50%
[Progresso: 300/4780]	Accuratezza corrente: 98.33%
[Progresso: 400/4780]	Accuratezza corrente: 98.75%
[Progresso: 500/4780]	Accuratezza corrente: 96.60%
[Progresso: 600/4780]	Accuratezza corrente: 93.67%
[Progresso: 700/4780]	Accuratezza corrente: 93.43%
[Progresso: 800/4780]	Accuratezza corrente: 91.62%
[Progresso: 900/4780]	Accuratezza corrente: 91.67%
[Progresso: 1000/4780]	Accuratezza corrente: 91.60%
[Progresso: 1100/4780]	Accuratezza corrente: 91.55%
[Progresso: 1200/4780]	Accuratezza corrente: 91.67%
[Progresso: 1300/4780]	Accuratezza corrente: 91.69%
[Progresso: 1400/4780]	Accuratezza corrente: 91.50%
[Progresso: 1500/4780]	Accuratezza corrente: 91.27%
[Progresso: 1600/4780]	Accuratezza corrente: 91.38%
[Progresso: 1700/4780]	Accuratezza corrente: 91.53%
[Progresso: 1800/4780]	Accuratezza corrente: 91.50%
[Progresso: 1900/4780]	Accuratezza corrente: 91.47%
[Progresso: 2000/4780]	Accuratezza corrente: 91.35%
[Progresso: 2100/4780]	Accuratezza corrente: 91.24%
[Progresso: 2200/4780]	Accuratezza corrente: 91.23%
[Progresso: 2300/4780]	Accuratezza corrente: 91.22%
[Progresso: 2400/4780]	Accuratezza corrente: 90.75%
[Progresso: 2500/4780]	Accuratezza corrente: 90.96%
[Progresso: 2600/4780]	Accuratezza corrente: 91.27%
[Progresso: 2700/4780]	Accuratezza corrente: 91.59%
[Progresso: 2800/4780]	Accuratezza corrente: 91.89%
[Progresso: 2900/4780]	Accuratezza corrente: 91.62%
[Progresso: 3000/4780]	Accuratezza corrente: 91.30%
[Progresso: 3100/4780]	Accuratezza corrente: 91.32%
[Progresso: 3200/4780]	Accuratezza corrente: 90.97%
[Progresso: 3300/4780]	Accuratezza corrente: 90.94%
[Progresso: 3400/4780]	Accuratezza corrente: 90.91%
[Progresso: 3500/4780]	Accuratezza corrente: 91.00%
[Progresso: 3600/4780]	Accuratezza corrente: 91.00%
[Progresso: 3700/4780]	Accuratezza corrente: 91.00%
[Progresso: 3800/4780]	Accuratezza corrente: 91.03%
[Progresso: 3900/4780]	Accuratezza corrente: 90.95%
[Progresso: 4000/4780]	Accuratezza corrente: 90.97%
[Progresso: 4100/4780]	Accuratezza corrente: 91.05%
[Progresso: 4200/4780]	Accuratezza corrente: 91.07%
[Progresso: 4300/4780]	Accuratezza corrente: 91.05%
[Progresso: 4400/4780]	Accuratezza corrente: 91.02%
[Progresso: 4500/4780]	Accuratezza corrente: 90.98%
[Progresso: 4600/4780]	Accuratezza corrente: 90.91%
[Progresso: 4700/4780]	Accuratezza corrente: 90.87%
[Progresso: 4780/4780]	Accuratezza corrente: 90.71%

```
=====
📄 REPORT FINALE DI VERIFICA SULLO ZIP
=====
Immagini totali elaborate:      4780
Predizioni corrette (Esatte):   4336
Accuratezza Globale (Accuracy): 90.71%
=====
```

```
In [50]: import warnings
from sklearn.metrics import balanced_accuracy_score, classification_report
import numpy as np

# Silenzia temporaneamente i warning di scikit-learn per y_pred/y_true sbilanciati
warnings.filterwarnings("ignore", message="y_pred contains classes not in y_true")

# =====
# 1. RECUPERO DATI E INVERSIONE DIZIONARIO
# =====
y_true = risultati_test[0]
y_pred = risultati_test[1]

TARGET_ID_TO_NAME = {v: k for k, v in TARGET_LABEL_MAP.items()}
id_classi_ufficiali = list(range(CONFIG["num_classes_target"]))

# =====
# 2. CALCOLO BALANCED ACCURACY
# =====
balanced_acc = balanced_accuracy_score(y_true, y_pred) * 100

print("=" * 60)
print(f"📊 BALANCED ACCURACY SULLE CLASSI PRESENTI: {balanced_acc:.2f}%")
print("=" * 60)

# =====
# 3. VERIFICA CLASSI MANCANTI
# =====
nomi_macro_classi = [TARGET_ID_TO_NAME[i] for i in id_classi_ufficiali]
classi_presenti_nel_disco = np.unique(y_true)
classi_mancanti = [TARGET_ID_TO_NAME[i] for i in id_classi_ufficiali if i not in classi_presenti_nel_disco]

if classi_mancanti:
    print(f"\n💡 Nota di Debug: Nella cartella 'archive' mancano le cartelle di")
    print(f"Il report mostrerà 0.00 di Recall su di esse perché non ci sono immagini")

# =====
# 4. CLASSIFICATION REPORT
# =====
print("📄 REPORT DETTAGLIATO PER OGNI MACRO-CLASSE:")
print(classification_report(
    y_true,
    y_pred,
    labels=id_classi_ufficiali,
    target_names=nomi_macro_classi,
    zero_division=0
))

# Ripristina i warning predefiniti per il resto del notebook
warnings.filterwarnings("default", message="y_pred contains classes not in y_true")
```

```
=====
🔗 BALANCED ACCURACY SULLE CLASSI PRESENTI: 89.79%
=====
```

💡 Nota di Debug: Nella cartella 'archive' mancano le cartelle di: ['Battery', 'Clothing', 'Organic', 'Undifferentiated']
Il report mostrerà 0.00 di Recall su di esse perché non ci sono immagini reali da testare.

📄 REPORT DETTAGLIATO PER OGNI MACRO-CLASSE:

	precision	recall	f1-score	support
Battery	0.00	0.00	0.00	0
Clothing	0.00	0.00	0.00	0
Glass	0.90	0.86	0.88	1002
Metal	0.89	0.91	0.90	820
Organic	0.00	0.00	0.00	0
Papery	0.99	0.94	0.97	1994
Plastic	0.88	0.88	0.88	964
Undifferentiated	0.00	0.00	0.00	0
accuracy			0.91	4780
macro avg	0.46	0.45	0.45	4780
weighted avg	0.93	0.91	0.92	4780

In []: